NUS Orbital 2026

Project Group:

F4C_Opossum_6748

Prepared by:

WONG BO XI FILBERT
LI JIANXI

Table of Contents

1. Executive Summary

This document serves as the official Milestone report for Project F4C Opossum, developed for the NUS Orbital 2026 programme. Aiming for the Apollo 11 level of achievement, our team is engineering a highly scalable, robust 2D side-scrolling strategy game using the Unity Engine and C#. This first milestone demonstrates a fully functional Proof of Concept (PoC) that encompasses the core game loop: resource economy, automated entity spawning, AI-driven adversarial generation, collision-based combat, and a definitive win/loss state via base destruction.

2. Project Vision & Motivation

2.1. Mission Statement

Inspired by the timeless mechanics of the classic flash game Age of War, F4C Opossum modernises the tug-of-war base defence genre. The game challenges players to balance a constantly ticking resource economy with tactical unit deployment to overwhelm a dynamic enemy AI.

2.2. Academic Motivation

As Computer Science undergraduates, we see this project as a deliberate push beyond standard university curricula. Building a game from scratch requires mastering complex, real-time software architectures, UI/UX interaction paradigms, and optimising intricate game loops. Targeting Apollo 11 requires us not only to deliver a fun game but also to adhere strictly to industry-standard software engineering practices, ensuring our codebase is clean, modular, and extensible.

3. Target Audience & User Stories

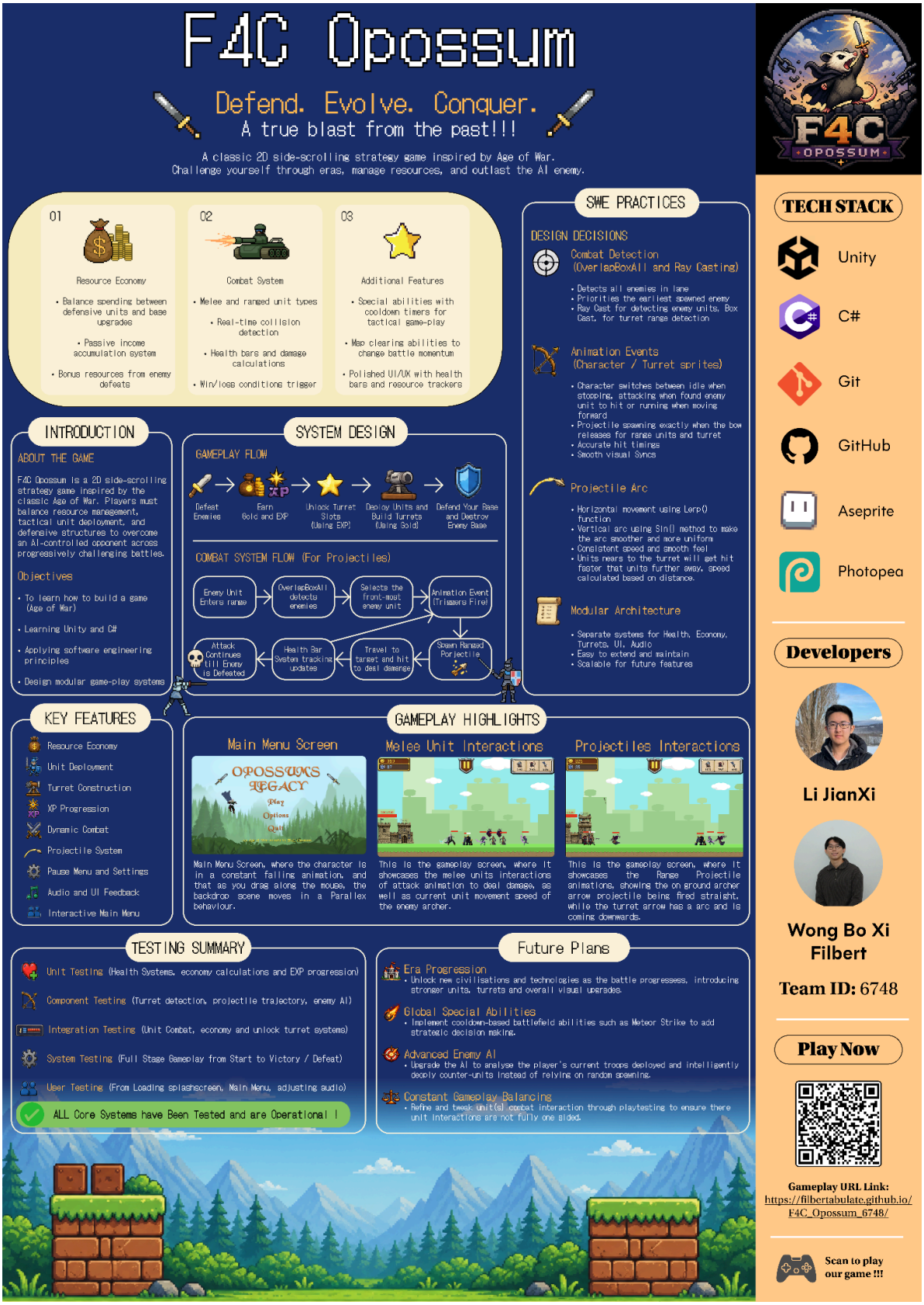
To ensure our development remains user-centric, we have defined four core player personas:

- **The Strategist:** “As a strategist, I want to balance my gold economy between spawning cheap frontline units and saving for expensive ranged units, so I can optimally counter the enemy’s formation.”
- **The Competitor:** “As a competitive player, I want the enemy AI to spawn units autonomously and unpredictably, so the game remains challenging and forces me to adapt my strategy on the fly.”
- **The Explorer:** “As an explorer, I want to see visual feedback when my units clash with the enemy, so I can clearly understand the combat mechanics and health systems.”
- **The Casual Gamer:** “As a casual player, I want an intuitive user interface with clear buttons and resource counters, so I can understand how to play within the first ten seconds of launching the game.”

4. Posters Created Throughout this Project

4.1.1. Liftoff and Milestone 1 Poster

4.1.2. Milestone 2 and 3 Poster



5. Core Game Features (Milestone 1)

To build a solid foundation for F4C Opossum, our initial milestones focused on establishing the core gameplay loop, entity interactions, and robust C# scripts. Currently, our Milestone 1 submission focuses on building the technical foundation necessary for the future gameplay systems. The current prototype demonstrates:

- A functioning 2D environment
- Unit spawning systems
- Movement and collision systems
- Base health management
- Combat interactions
- Dynamic HP bar rendering

These features help to establish the minimum viable gameplay loop required for future expansion.

5.1. Basic 2D Engine setup

To start the project, we had to set up the overview of how the gamemap would look, as well as the layers we had to consider when building the map. For example, different layers like (foreground, default, background), as well as layers for unit tagging to differentiate between allied units and enemy units.

As we are planning to build an Age of War game based on a 2D side-scrolling game, we decided to utilise the Unity Universal 2D template.

5.2. Entity Spawning System

Since Age of War is a 2D Player versus Enemy game that requires user input to spawn units against AI/bot enemies, we needed to create spawning scripts for both player-controlled and enemy-controlled units.

Current Functionality:

- Dynamic runtime instantiation
- Separate spawn points
- Independent player/enemy spawning systems

Scripts Created for Spawning System:

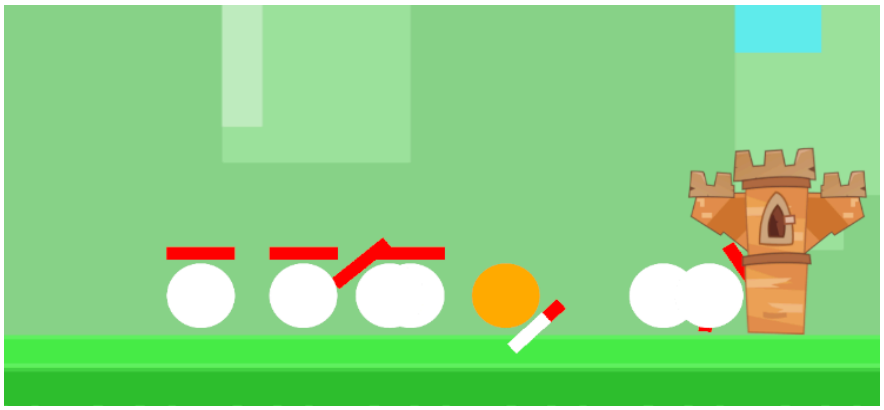
- PlayerSpawner.cs
- EnemySpawner.cs

Design Configurations:

- Player unit prefabs are loaded dynamically from a “Resources/PlayerUnits” folder at startup.
- The player (user) can deploy units by pressing the “Z” key.
- Originally, a 0.5-second cooldown was enforced on player spawning to prevent units from jittering down and improperly stacking. However, since we had factored in the idea that the unit stacking is alright for allies only at the base, the original jittering issue has been resolved, where the cooldown now is to simulate the time it takes to “train” up the unit.
- Enemy units are similarly loaded from a “Resources/EnemyUnits” folder.
- Enemies currently spawn automatically on a 25-second timer.
- Both spawners actively monitor the battlefield and will halt all spawning if their respective base’s HealthSystem is destroyed.
- A sequential spawnOrder tracker tags every instantiated unit, allowing the game to distinguish older units from newer ones. This feature prevents unit stacking bugs, as stacked units now know which unit should move first.

Challenges faced during the building process of this feature:

One issue encountered was unit overlap during spawning. This occasionally caused units to collide incorrectly or become stuck during movement. This was partially resolved by implementing collision checking and movement spacing between allied units. The image below shows the units colliding during testing of the collision mechanics. In the previous version, the enemy unit spawn was too close to the map, causing enemy units to spawn on “ally” units and creating this undesired bug.



Design Decisions and Future Considerations:

We chose to create two separate scripts, one for ally spawning and one for enemy spawning, since we know that enemy spawning is initiated by the

Computer code, so there is no need to add features that read user input to spawn units. On the other hand, since the player units are spawned not based on time but by user input, we need to factor the reading of user inputs into the PlayerSpawner.cs script.

However, future considerations include adding features like monetary tracking so that units can only be spawned when the player/enemy has enough money to deploy them, as well as creating buttons to deploy units instead of using keycap values for players. That way, the user can see what the unit they are deploying looks like before choosing to deploy it.

5.3. Combat & Movement AI

The combat and Movement feature allows units to move automatically toward enemy targets and engage in combat once enemies enter range. The units are also able to do smart switching between enemy units and the enemy base, where they will switch targets to the earliest spawned unit to attack if they were initially attacking the enemy base when this enemy unit was spawned.

Current Functionality:

- Automatic movement
- Enemy detection
- Attack cooldown systems
- Damage application
- Base destruction logic

Scripts Created for Unit Movement and HP interactions:

- UnitMove.cs
- HealthSystem.cs

How the Combat System works:

- Units begin in an undeployed state and must fall to physically hit the ground layer (detected via Physics2D.OverlapCircle) before they can begin marching.
- Movement direction is determined dynamically by Unity tags; players move right (positive X direction), while enemies move left (negative X direction).
- **Ally Collision:** Units utilise 2D Raycasts (Physics2D.RaycastAll) filtered by layer masks to detect allies directly ahead of them. A unit will stop and wait if the ally in front of it has an older spawnOrder, effectively preventing units from clipping through or passing each other.

- **Targeting:** Units use dynamic bounding boxes (Physics2D.OverlapBoxAll) to detect targets within their specific attack range. The AI scans the overlapping zone and prioritises attacking the oldest spawned enemy unit first. If no enemy units are alive, the AI seamlessly switches its target to the enemy base.
- Combat operates on a 1-second attack cooldown. (This is a placeholder value which can be customised for different units in the future)
- Units deal their predefined damage values directly to the target's HealthSystem once the cooldown completes.

Design Decisions and Future Considerations:

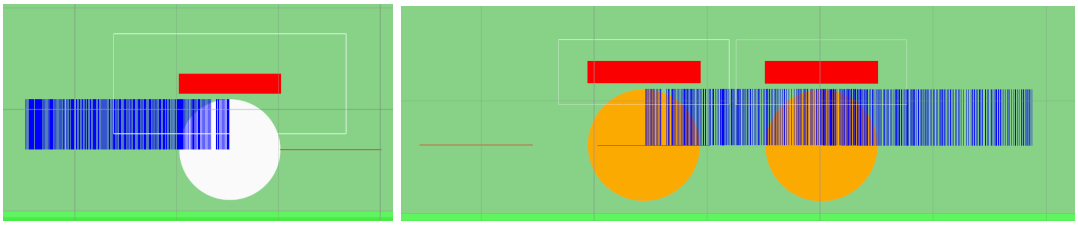
The current combat system design has multiple functions that handle the following:

- Movement logic
- Health management
- Attack behaviour

However, all these functions are currently in a single file, UnitMove.cs. As such, future considerations include breaking these functions into new files to keep them clean and more meaningful.

Challenges faced during the building process of this feature:

Throughout the process of creating, there were a lot of trials and errors in fine-tuning the unit collision detection for both allies (RayCast) and enemies (BoxAll) to ensure no unintended behaviour would occur during gameplay testing. The image below shows the debugging steps and trackers we place to see how the combat collision detection system is used, with the red line indicating the unit's "sight" in front to determine whether the unit in front is an ally or a foe. The blue line is all objects it was colliding with, which we found out was because it was tracking the collision with itself as well.



5.4. Modular Health & UI System

Following a Dynamic HP bars template from YouTube tutorials, we implemented it for both bases and combat units. We also fine-tuned the template, allowing the

Health Points bar to drop vertically or horizontally so the system accommodates both orientations.

Current Functionality:

- Real-time HP updates
- World-space UI tracking
- Visual combat feedback

Scripts Created for Health Bar UI:

- HealthBarUI.cs
- HealthSystem.cs

How the Health and UI System works:

- We developed a modular HealthSystem component to manage hit points, with a default maximum of 100 HP (temporary value). Currently, bases have 500 HP, enemy units have 100 HP, and ally units have 150 HP.
- The system takes damage, clamps health values to prevent bugs, and automatically destroys the GameObject when HP reaches zero.
- The system is decoupled from the HealthBarUI script, which visually represents health percentages by adjusting a RectTransform's size.
- The UI supports horizontal orientations for standard units and vertical orientations for the bases.

Challenges faced during the building process of this feature:

Initially, a challenge was how to get the HealthBar to move with the units rather than remain static. since the tutorial did not really cover assigning the health bar to a unit. After searching for a bit, the fix was to put the health bar, in this case the "Canvas UI", as a child of the UI; that way, when the unit moves, the health bar moves along with the unit.

5.5. Dynamic Camera System

Originally, we created the Gamemap layout to fit only the current camera point of View, with the entire map situated nicely within the camera boundaries. However, during testing, we felt that the "runway" for units to move was too short. As a result, to expand the map, we also need to ensure that the camera can move freely across the map without needing to be moved by following a player unit (as covered in the Mission Control 2 Webinar), but by following the user's mouse cursor instead.

Therefore, this feature is designed to let a camera controller keep the battlefield visible throughout gameplay, allowing the user to see different parts of the map.

Scripts Created for Health Bar UI:

- CameraController.cs

How the Camera Controller movement works:

- Players can pan the camera horizontally by moving the mouse cursor within 50 pixels of the edges of the screen.
- The camera's movement is clamped to predefined background boundaries to keep the focus on the battlefield.
- Users can utilize the mouse scroll wheel to zoom in and out.
- The zoom, which utilises a mouse scroll wheel, is clamped between a minimum and maximum value, and the camera is mathematically anchored to ground level, so zooming out reveals more of the map rather than exposing underground areas.

6. Future Roadmap (Milestones 2 & 3)

With the MVP architecture secured, development will now pivot to expanding complexity, balance, and visual fidelity.

6.1. Milestone 2 Priorities

- **Monetary Systems with Buttons to Spawn Units:** Implement a currency tracker that tracks the player's current balance to identify which units the player can summon and which units the player is unable to summon. This includes earning money by defeating enemy troops, having a base starting amount, and potentially having a "turret" that, instead of attacking enemy units, helps earn "coins."
- **Art Integration:** Replace placeholder geometry with fully animated 2D sprites (idle, walk, attack, and death animations).
- **Implementation of Turret Defence:** Since our Bases right now are quite empty, with nothing to protect them, we need to implement slots that can be unlocked via the currency, as well as turrets that can be placed on the unlocked slots to deal damage to enemy when within their range or do passive abilities, like gaining currency.

6.2. Milestone 3 Priorities

- **Special Abilities:** Integrate cooldown-based global abilities (e.g., calling down a meteor strike on the battlefield).
- **Advanced AI:** Upgrade the EnemySpawner script to read the player's unit composition and intelligently spawn counter-units, rather than relying on randomised timers.
- **Audio Engineering:** Add background tracks, UI interaction clicks, and combat sound effects.
- **Polish & CI/CD:** Finalise game balance, conduct extensive bug testing, and potentially set up automated web-GL deployment via GitHub Actions.

7. Modern Technical Stack

F4C Opossum was built using industry-standard tools to ensure a high-quality end product.

- **Development Engine:** Unity (C#) was chosen as our primary engine because it is the industry standard for handling high-performance 2D game loops and physics calculations.
- **Art & Assets:** We utilised Aseprite as our precision tool for creating custom 2D pixel art assets and animations, supplemented by consistent, professional visuals sourced from Kenney.nl.
- **Project Management:** We used Git & GitHub for version control (employing a feature-branch workflow with peer reviews), and Kanban boards for agile sprint tracking to maintain transparent progress.

8. Software Engineering Rigour

To keep our codebase scalable and maintainable, we applied several established design patterns and strict encapsulation:

- **State Pattern:** Applied to manage overall game phases (Menu, Playing, Paused) as well as the distinct behavioural states of individual units on the battlefield.
- **Observer Pattern:** Used extensively to decouple the UI elements (such as health bars managed by HealthBarUI and resource counters) from the underlying core game logic (managed by HealthSystem), ensuring that visual updates do not bottleneck game performance.
- **OOP Excellence & Encapsulation:** We enforced strict object-oriented programming principles across all scripts. For example, critical tracking variables like spawnOrder are strictly encapsulated with the [SerializeField] attribute,

making them inaccessible to other scripts while still visible in the Unity Inspector for debugging.

9. Methodology & Quality Assurance

Consistent iterative development and testing were paramount to our success.

- **Agile Sprints:** We operated in two-week development cycles, enabling continuous, iterative feedback and rapid pivoting when mechanics required rebalancing.
- **CI/CD Workflow:** We strictly enforced a feature-branching strategy with mandatory Pull Requests (PRs), which effectively prevented regressions from breaking our main branch.
- **Testing Strategy:** We dedicated significant time to extensive bug testing and code refactoring, resulting in a highly polished user experience.

10. Conclusion for Milestone 1

Our Milestone 1 stage successfully establishes the project's technical foundation through a functional combat prototype and modular gameplay systems. The current implementation demonstrates the project's feasibility and provides a scalable architecture for future gameplay extensions, such as AI progression systems, economy management, and special abilities.

11. Source Code Documentation & Engineering Notes

The GitHub repository contains detailed inline code comments and engineering notes explaining key gameplay systems, implementation decisions, and debugging considerations throughout development.

These comments were added to improve:

- Maintainability
- Scalability
- Readability
- Debugging efficiency

Key systems documented include:

- Combat and collision systems
- Movement state handling
- Raycast detection logic

- Spawn management systems
- HP synchronisation systems

As for where the relevant scripts covered in this document can be found, they are under the “Assets/Scripts/” folder.

11.1. GitHub Repo Link

https://github.com/Filbertabulate/F4C_Opossum_6748

11.2. Video Gameplay for Milestone 1

https://drive.google.com/file/d/1kpUUDA6sRlNWCUQp549kR7V0Ms-xXScX/view?usp=sharing

11.3. Overall Project Logs

https://docs.google.com/spreadsheets/d/1UeVmOn5maOyk0yjt6T7f1WFaAEKY951M/edit?usp=sharing&ouid=107132362133645130135&rtpof=true&sd=true

12. Milestone 2: Expanding Complexity and Polishing

Building upon the MVP architecture secured in Milestone 1, development for Milestone 2 pivoted toward expanding game complexity, refining visual fidelity, and implementing the planned economy and defence systems. This phase introduced significant challenges in asset integration and required robust debugging of movement logic to maintain a seamless gameplay loop.

12.1. Art Integration, Animation, and Asset Challenges

A major priority for this milestone was replacing placeholder geometry with fully animated 2D sprites (idle, walk, and attack animations). Sourcing and integrating assets from different artists also introduced several technical and visual inconsistencies that required systematic debugging.

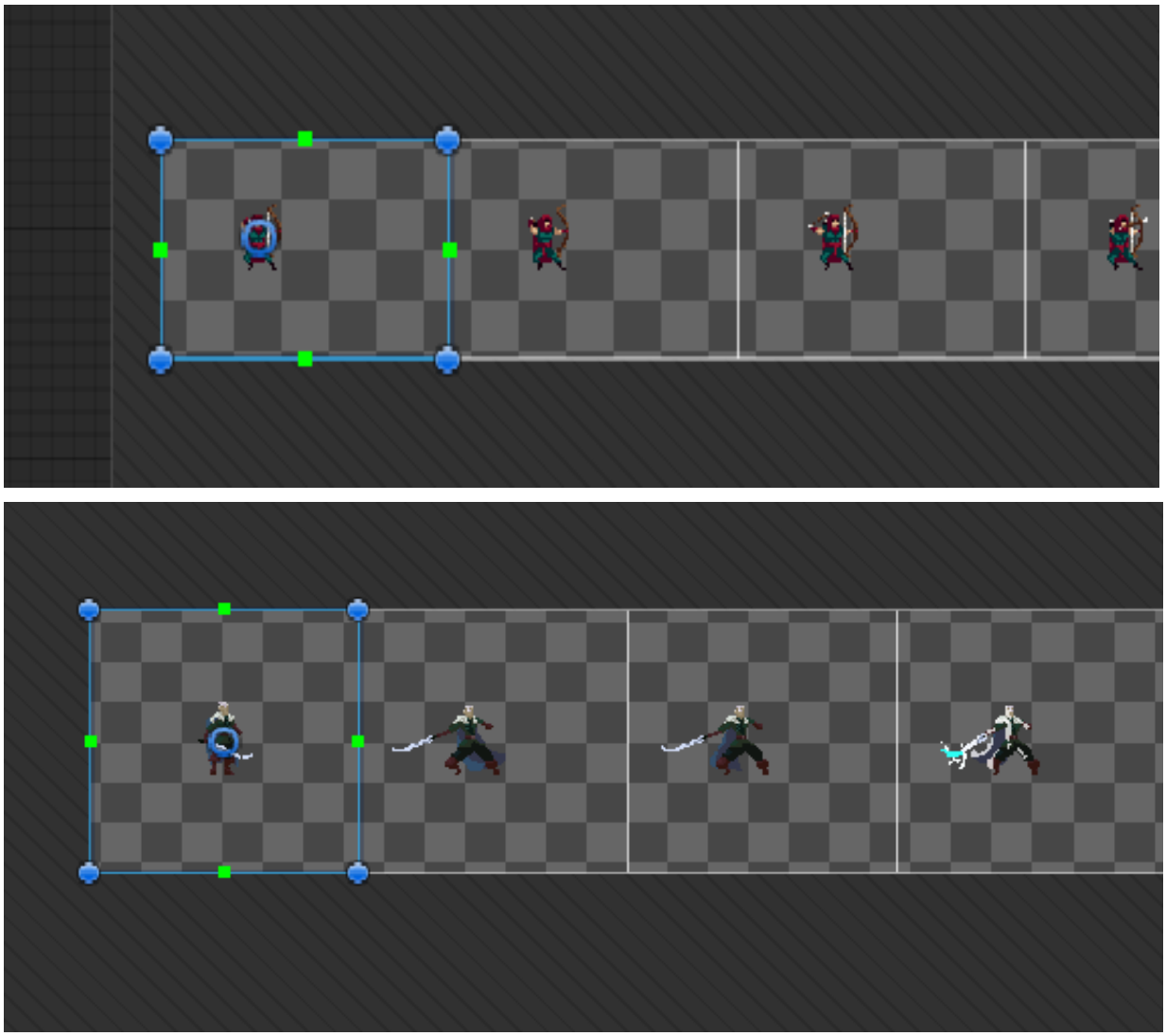
12.1.1. Pixel Per Unit (PPU) Inconsistencies:

Assets sourced from different creators often had vastly different pixel densities. This caused units to spawn at drastically different scales in the Unity engine. We standardised the PPU across all imported textures to ensure consistent scaling.

12.1.2. Pivot Point Misalignments:

Different artists utilise different centre anchor points for their sprites (e.g., bottom-centre vs true centre). When instantiating units dynamically, this discrepancy

caused certain units to spawn slightly underground or float above the ground layer. We resolved this by meticulously standardising the pivot points in Unity's Sprite Editor for all unit frames.

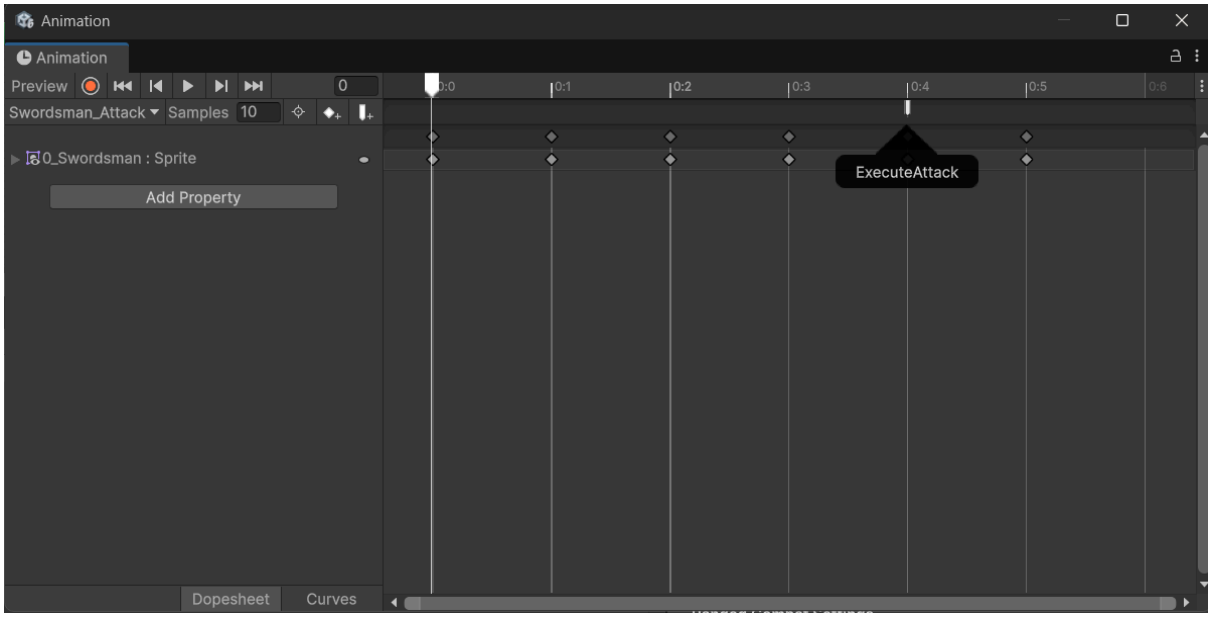


Note: On the top is the player Archer unit attack animation frame by frame, while on the bottom is the player magic knight unit attack animation frame by frame

12.1.3. Animator Integration & Animation Events:

To ensure combat visual feedback aligned accurately with the underlying damage calculations, we heavily utilised Unity's Animator component. A significant challenge was ensuring units did not deal damage before their attack animation physically connected with the enemy. To resolve this issue, we decoupled the attack timer from the direct damage application. Instead, the timer triggers the AttackTrigger in the

Animator, and an Animation Event embedded directly into the sprite's attack frame calls the ExecuteAttack() method.



Note: Unity Animation panel that allows additions of event triggers at a certain frame of the animation to make it look smooth.

12.1.4. Lack of Free Assets and 2D Sprites:

When embarking on this project, we initially underestimated the sheer volume of artistic resources required to bring a game to life. While writing the backend logic and C# scripts presented its own deep technical challenges, establishing a compelling visual identity proved to be one of our most significant bottlenecks.

One of the most persistent issues we faced was the scarcity of high-quality, free-to-use digital assets and 2D sprites. Because F4C Opossum is an educational, zero-budget project intended to be entirely free-to-play, purchasing premium asset packs was simply out of the question.

Consequently, we had to rely heavily on open-source platforms. We spent a tremendous amount of time and energy scouring the internet just to find characters, enemies, and environments that actually looked like they belonged in the same universe. Trying to stitch together artwork created by completely different artists, each utilising their own unique colour palettes, shading techniques, and pixel densities, was an exhausting process. It took painstaking effort to ensure our game did not end up looking like a disjointed, visual patchwork, and required us to heavily edit and standardise the assets we did manage to find.

We also experimented with AI image generation tools to create certain pixel-art assets when suitable free alternatives are less desirable. However, the AI-generated sprites rarely integrate seamlessly into our game. This is because they often varied in proportions, animation style, colour palette, and pixel density, making them visually inconsistent with the rest of our assets.

Moreover, even when the AI generates a sprite sheet, we have to take that sprite sheet to a free photo editing platform like Photopea to crop out each image, making sure that each background is transparent manually, before needing to stitch them back up again from individual PNG images. Although this workflow eventually produced somewhat usable assets, the repetitive manual editing significantly increased the development time for even a single animated character.

Ultimately, navigating this logistical nightmare gave us a profound, newfound appreciation for the game development industry at large. Experiencing firsthand the staggering amount of effort it takes just to source, format, and animate a handful of 2D pixel-art sprites for a student prototype really highlighted the monumental labour required by entire departments of conceptual artists, riggers, and technical animators to develop visually breathtaking AAA titles like Elden Ring.

12.2. Economic System

12.2.1. Current Functionality:

To deepen the strategic layer, we implemented the planned monetary system, requiring players to balance resource generation against unit deployment costs. The UI was completely overhauled to display current Gold, Experience Points (EXP), and specific unit deployment buttons.

12.2.2. Scripts Modified

- PlayerSpawner.cs

12.2.3. How the Economic System Works

Currently, a passive income generator is being used in the Player Spawner script, providing a baseline economy. The UI buttons were refactored to pass a specific unitIndex parameter, ensuring players can make tactical choices rather than relying on randomised spawning.

- 50 Gold at the start, gain 5 Gold per game second
- 0 EXP at the start, gain 1 EXP per game second

12.3. Advanced Unit Formation and Movement Animation

12.3.1. Current Functionality

Building upon the combat system developed in Milestone 1, the movement animation was significantly enhanced to allow units to maintain smooth marching formations. Rather than simply stopping whenever another allied unit was detected in front, units are now capable of dynamically matching the movement speed of the unit they are following. This prevents large formations from constantly stopping and starting, resulting in much smoother gameplay and more natural-looking marching animations.

12.3.2. Scripts Modified

- UnitMove.cs
- EnemySpawner.cs

12.3.3. How the Movement System Works

Originally, the previous implementation relied only on an ally detection raycast positioned in front of each unit. Whenever another allied unit entered this detection zone, the trailing unit would immediately stop until the path became clear.

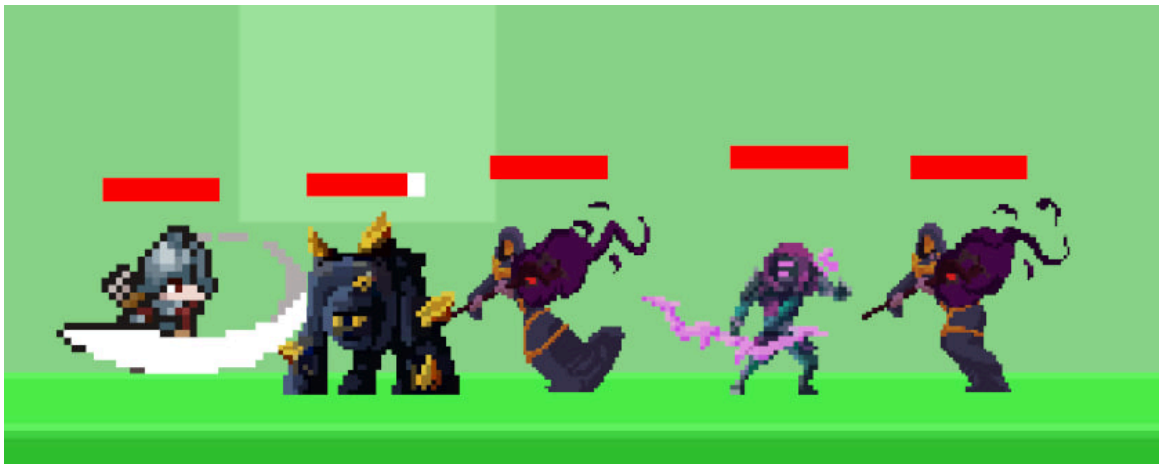
The updated implementation introduces a second tracking system that continuously monitors the ally directly ahead. Once a unit resumes movement, it compares its own movement speed against the currently tracked ally (the one directly in front of the current unit) and temporarily adopts the slower movement speed whenever necessary. By synchronising movement speeds instead of repeatedly stopping, units can maintain a consistent pace while moving together.

This behaviour also allows attack animations and walking animations to remain synchronised across large groups of units, significantly improving the overall visual quality of combat.

12.3.4. Challenges faced during the building process of this feature:

One major issue encountered was incorrect ally detection. During the initial implementation, units only relied on a single forward-facing detection ray. As units constantly entered and exited this raycast, they frequently locked onto

the wrong ally, causing units to repeatedly stop, accelerate, and / or switch targets. This resulted in noticeable animation glitches and jittering movement.



Note: As shown, the enemy units' fire mage is still constantly running behind the golem, even though the enemy golem is currently engaging in battle and is not moving any more.

To resolve this issue, as mentioned, a separate ally tracking system was implemented alongside the original forward detection logic. The forward raycast determines when a unit should stop, while the tracking system continuously follows the specific ally directly ahead, allowing smooth speed synchronisation without unnecessary movement interruptions.

12.4. Main Menu and Environmental Effects

12.4.1. Current Functionality:

To improve the player's first impression of the game, a fully interactive main menu was created. The menu incorporates mouse-driven parallax effects, animated environmental elements, and a continuously animated hero character to create a more dynamic title screen.

12.4.2. Scripts Created

- MenuParallaxBehaviour.cs
- Animator Controllers (Fall.anim)

12.4.3. How the Main Menu System Works

With this implementation, the foreground and background objects respond to the user's mouse position using smooth interpolation, creating a layered parallax effect that gives the menu additional depth. Environmental assets (By Raycastly), such as trees and grass, are animated simultaneously using

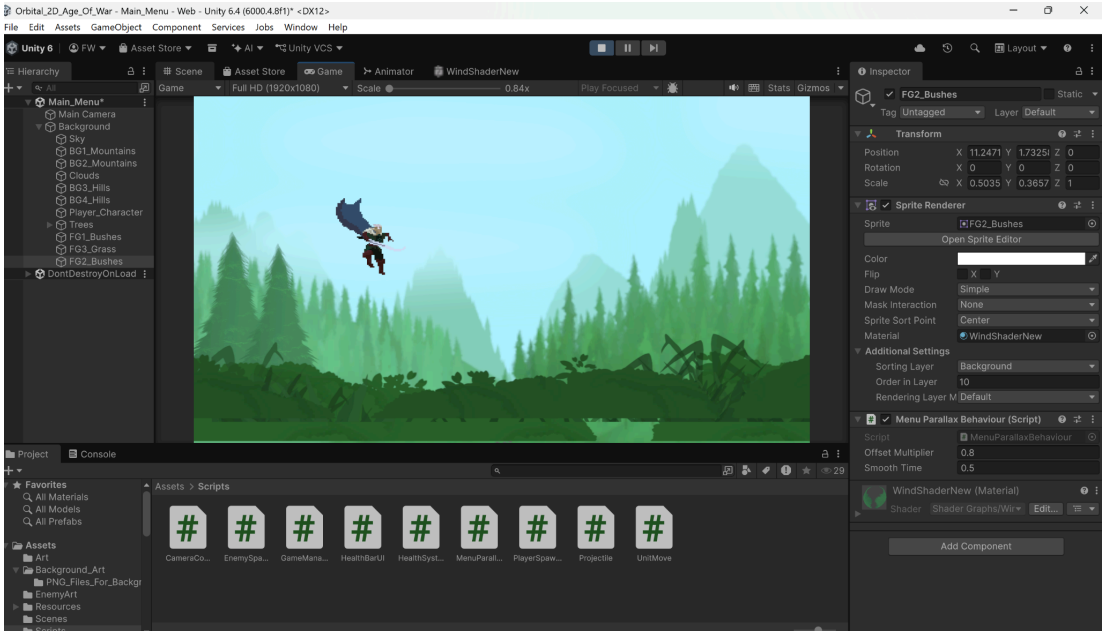
shader-based wind effects, all while the hero character continuously loops a falling animation.

12.4.4. Challenges faced during the building process of this feature:

Achieving a natural-looking environment required a considerable amount of fine-tuning. Initially, the cape animation of the hero moved at a noticeably different speed from the surrounding wind animations applied to the trees and grass. Although each animation functioned independently, the mismatch reduced the overall realism of the scene.

After multiple trial and error, the playback speed of the character's animation was adjusted to better match the movement speed of the environmental wind shader, creating a much more visually cohesive main menu.

Another issue involved the foreground grass used in the parallax system. Large mouse movements caused the grass layer to shift too far, exposing the background beneath the map. This was resolved by enlarging the foreground grass sprite while simultaneously reducing the overall parallax movement multiplier.



Note: The error shown above is the grass being exposed when the mouse movement is too large, causing it to be exposed. (A considerable amount of time and effort was spent on placing each tree, grass and all other components in this current backdrop).

12.5. Scene Transition System

12.5.1. Current Functionality:

To create smoother transitions between gameplay scenes, both cross-fade and circular fade (Not being used at the moment, but is a good thing to have for potential future implementations) transition effects were implemented. These transitions are utilised throughout the game when we are switching between the gameplay scene and the main menu scene.

12.5.2. Scripts Created

- LevelManager.cs
- SceneTransition.cs
- CrossFade.cs
- CircleWipe.cs

12.5.3. How the Transition System Works:

Scene loading is controlled through a central LevelManager, which triggers different transition animations depending on the destination scene. The transition animation plays before asynchronously loading the next scene, resulting in a seamless visual experience. Moreover, from the main menu screen, when play is clicked, the transition loads with a progress task bar being shown to showcase the progress between transiting the main menu to the gameplay mode.

12.5.4. Challenges faced during the building process of this feature:

One issue encountered was regarding the circular fade transition. Initially, the expanding circle never completely covered the screen because its size was calculated relative to the world space rather than the UI Canvas.

Note: Circle wipe transition does not fully cover the screen at first

How we managed to fix this issue is by increasing the maximum radius of the transition sprite while anchoring the animation relative to the Canvas itself. This ensured that every transition fully covered the display, regardless of screen resolution.

12.6. Audio Management System

12.6.1. Current Functionality:

A global audio management system was implemented to control background music, button click effects, button hover effects, and user-configurable volume settings across every scene in the game.

12.6.2. Scripts Created:

- AudioSettingsUI.cs
- MusicLibrary.cs
- MusicManager.cs
- SoundLibrary.cs
- SoundManager.cs

12.6.3. How the Audio System Works:

A persistent AudioManager, i.e., the MusicManager and SoundManager, is maintained across scene changes using Unity's singleton pattern. Between scenes, if there is a change in audio volume, it would tie back to the Manager instance to change volume, background music for MusicManager and SFX music for Sound Manager.

12.6.4. Challenges faced during the building process of this feature:

The main challenge was trying to enforce the background music to be constantly played after changing scenes. Without setting up a persistent audio manager, Unity recreated new AudioSources every time a scene loaded, causing the music to restart repeatedly. Therefore, I had configured the game to have the audio manager at the main menu screen, since this is the very first scene that will be loaded for the users.

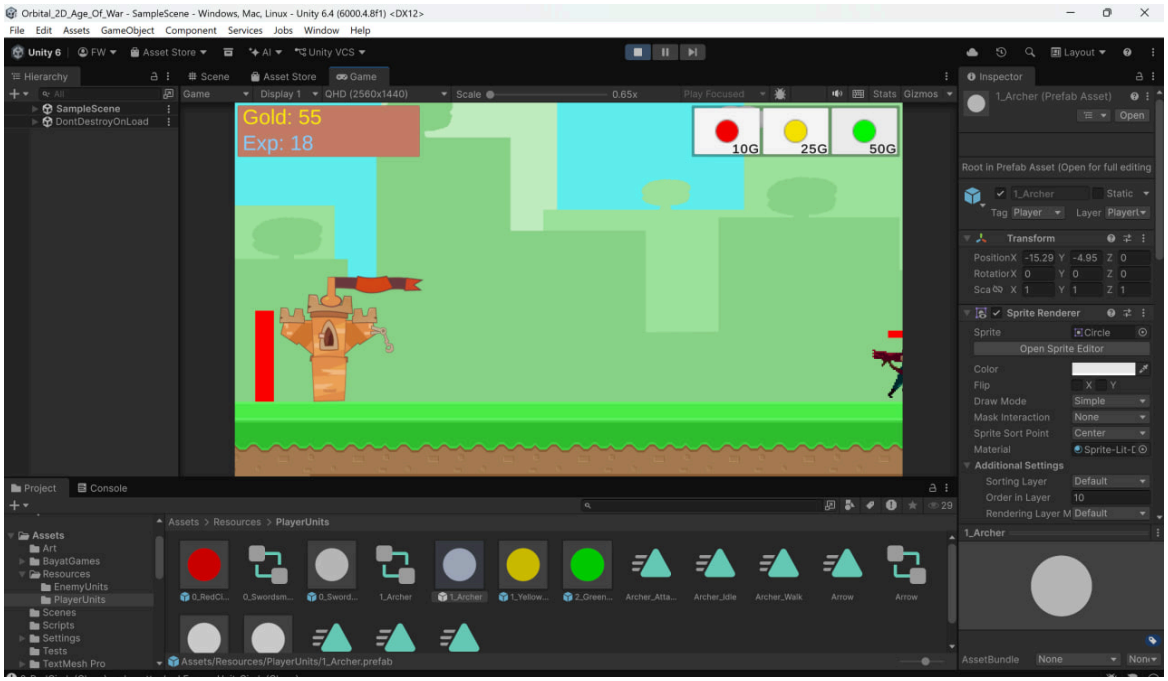
12.7. User Interface Refinement

12.7.1. Current Functionality:

To improve visual clarity, the interface for purchasing units, unlocking turret slots, and displaying player resources was redesigned using custom pixel-art assets (Mainly AI generated since we could not really find UI templates that match what

we want). New icons for Gold, Experience Points, turret slots, and deployment buttons, castle sprites were introduced to better match the overall medieval aesthetic of the game.

Before UI Refinement:



After UI Refinement

12.7.2. Challenges faced during the building process of this feature:

The main issue was just finding UI assets that showcase the theme we are going for. Considerable time was spent editing, resizing, and standardising these assets to create a consistent user interface.

12.8. Pause Menu System

12.8.1. Current Functionality:

A pause system was implemented, allowing players to pause gameplay, adjust settings, resume the game, or return to the main menu.

12.8.2. Scripts Created:

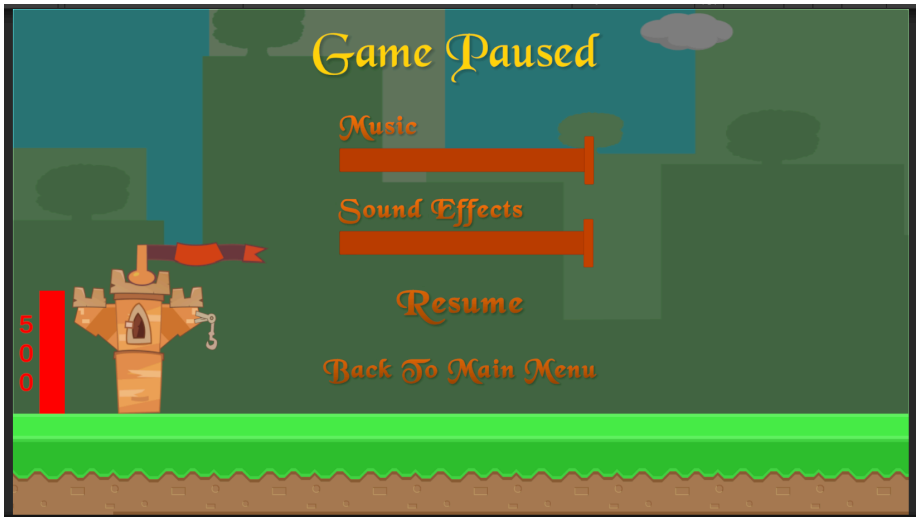
- PauseManager.cs

12.8.3. How the Pause System Works:

Activating the pause menu sets Unity's Time.timeScale to zero while displaying a full-screen overlay containing navigation buttons and audio settings.

12.8.4. Challenges faced during the building process of this feature:

Initially, the pause menu failed to properly cover the gameplay scene. Certain interface elements also disappeared whenever the game window was resized.



Note: As shown, the grass and tower, as well as the HP bar, are not being covered by the translucent background even though the game pause menu is activated.

Researching online, it turns out the main issue was that the Canvas had been configured using Screen Space - Camera mode. Migrating the UI to Screen Space - Overlay mode resolved the rendering order issues. Following this migration, all interface elements were manually repositioned and resized to ensure correct behaviour across different screen resolutions.

12.9. Turret Construction and Defence System

12.9.1. Current Functionality:

A modular turret construction system was developed, allowing players to first unlock turret scaffolding (using EXP) before constructing fully operational defensive towers (Using Gold). Once deployed, turrets automatically detect nearby enemies and fire projectiles whenever targets enter their attack range.

12.9.2. Scripts Created:

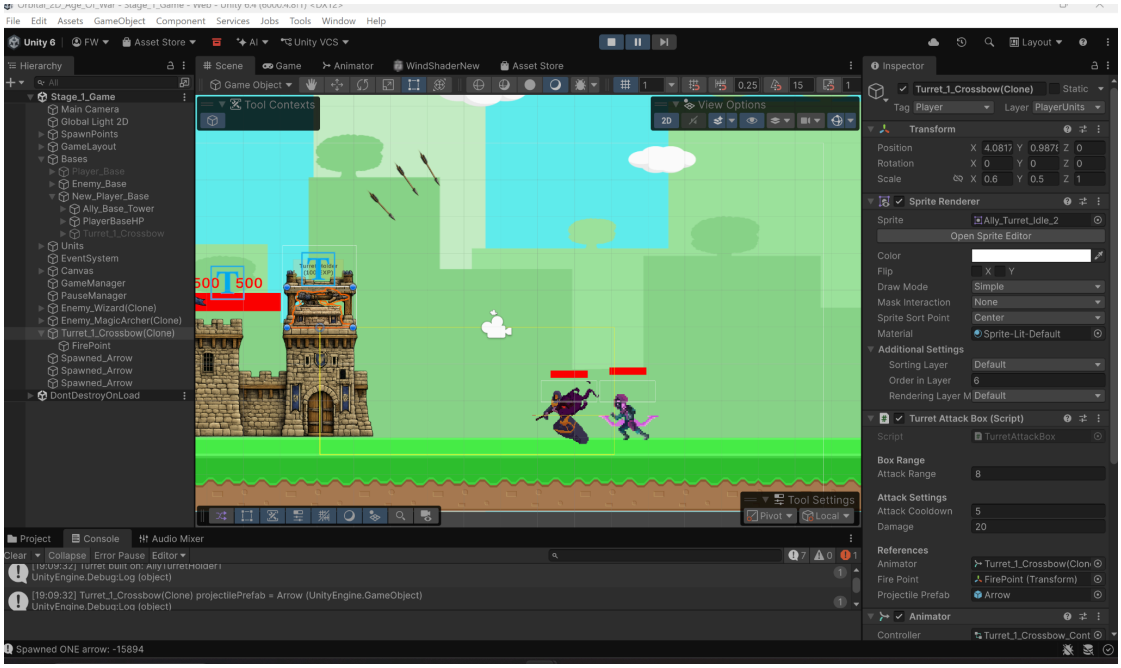
- TurretSlotUI.cs
- TurretAttackBox.cs
- TurretArcProjectile.cs

12.9.3. How the Turret Works:

Each turret continuously scans its attack range for enemy units. Upon detecting a valid target, the turret rotates towards the enemy, plays its attack animation, and launches a projectile towards the selected target. The scanning method uses a BoxAll method to find the earliest spawned enemy that enters that target box, locking on to that target until it is destroyed to lock on to another unit

12.9.4. Challenges faced during the building process of this feature:

A particularly difficult bug involved multiple arrows being fired from a single attack animation. Upon revisiting all my configurations, it turns out that the attack animation had Loop Time enabled within Unity's Animator component. As a result, Unity repeatedly replayed the attack animation before transitioning back to the idle state, causing multiple projectile spawn events to trigger.



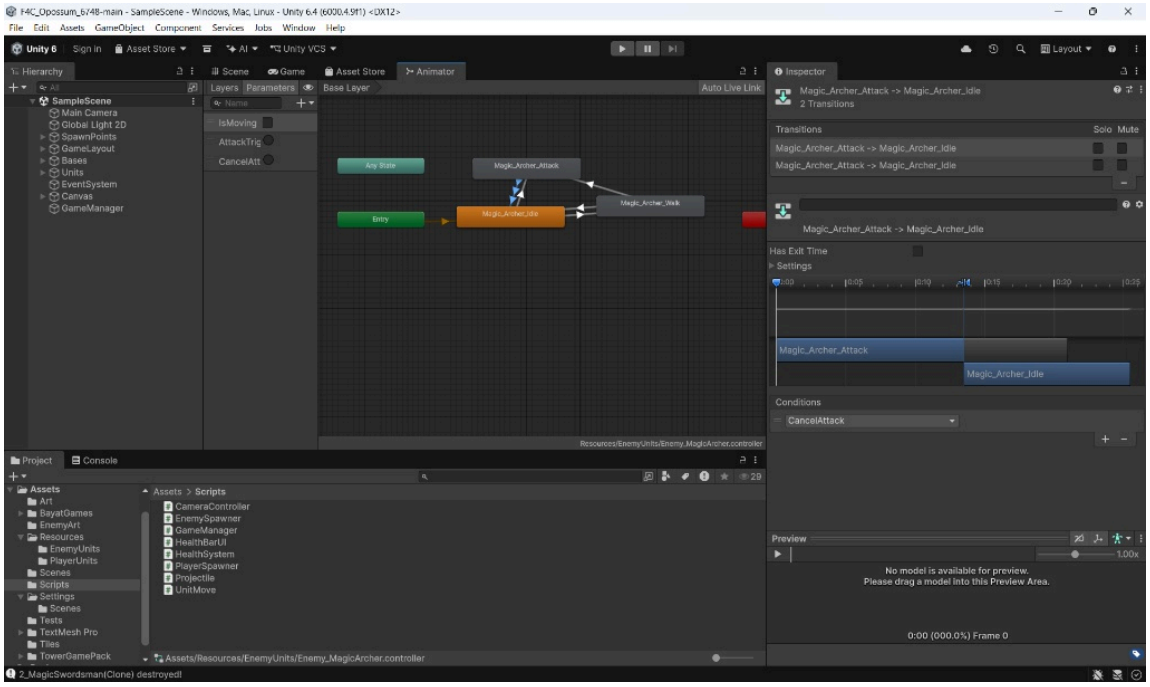
Note: Multiple arrows are being shot from the crossbow turret all at once, even though the console log tracks only one shot should have been fired. In this picture, we are also able to see how the turret attack range is like via the faint yellow box, which is it attackable area.

Disabling Loop Time for the attack animation ensured that only a single animation event was executed during each attack cycle, guaranteeing exactly one projectile per attack.

Other Notable Difficulties Faced Throughout Milestone 2

12.10. Unity Animator State Machine

While integrating the 2D sprites, one of the most significant technical hurdles we faced was understanding and implementing Unity's Animator component. Transitioning from pure C# logic to a visual Finite State Machine (FSM) introduced several unexpected synchronisation issues between our code and the visual output.



- **State Machine Complexity:** As seen in the configuration for the Enemy_MagicArcher.controller, we had to design a robust flow between Entry, Idle, Walk, and Attack states. Learning how to properly map script variables to Animator Parameters (IsMoving boolean, AttackTrig trigger, and CancelAtt trigger) required extensive trial and error. Some common bugs faced when configuring these settings are the characters being stuck in their idle animations or walking perpetually without attacking enemy units.

- **The "Has Exit Time" Problem:** Initially, our units felt sluggish and unresponsive during combat. We discovered this was caused by Unity's default Has Exit Time setting on transitions. Units were waiting for their idle or walk animations to finish 100% of their cycle before transitioning to an attack. Disabling Has Exit Time and relying purely on condition triggers (AttackTrig) made combat feel instantly responsive.

- **Handling Interrupted States:** An edge case emerged when a target was destroyed while a unit was mid-swing in its attack animation. If the target died, the unit would remain stuck in the attack loop because the FSM lacked an escape route. To solve this, we engineered an emergency interrupt system using the CancelAttack trigger. As shown in our Animator setup, this condition forces an immediate transition from the Attack state back to Idle, allowing the UnitMove.cs script to safely re-evaluate its next target without animation lock-up.

12.11. Version Control with Unity

This was one of the biggest challenges that we faced as a team, which is not typically encountered in standard web or software development. Managing Git version control with Unity's proprietary file structures caused a massive headache, as Unity scenes (.unity) and prefabs (.prefab) are serialised as massive text files. Early in Milestone 2, concurrent edits to the SampleScene resulted in merge conflicts that corrupted the scene hierarchy. We often had to deal with merge conflicts which resulted in a lot of time wasted pulling, pushing, and figuring out what went wrong and how we can fix it. It had occurred a total of 3 times during Milestone two, and we spent a total of 18 hours fixing these git merging issues.

To resolve this, we took turns working on the main scene and individually, we worked exclusively on isolated Prefabs (e.g., updating the MagicArcher prefab in isolation). This ensured that Unity's Hidden Meta Files setting was explicitly managed, guaranteeing that no missing script references or broken GUIDs occurred during branch merges.

12.12. Conclusion for Milestone 2

Milestone 2 represents a significant progression from the technical proof of concept established during Milestone 1 into a fully playable game prototype. Throughout this development cycle, we successfully integrated the planned economy system, fully animated combat units, modular turret defence mechanics, polished user interface elements, audio management, and seamless scene transitions into a cohesive gameplay experience.

Beyond implementing new gameplay features, a considerable portion of this milestone was dedicated to improving the overall quality and maintainability of the project. Numerous bugs relating to unit movement, animation synchronisation, UI rendering, scene transitions, and asset integration were identified and systematically resolved through extensive gameplay testing and iterative refinement. These debugging efforts not only improved gameplay smoothness but also strengthened our understanding of Unity's animation system, scene management, and software engineering practices.

Overall, we feel that the project has now progressed beyond demonstrating isolated mechanics into a functional prototype that closely resembles the intended final product. The current architecture provides a stable and extensible foundation upon which additional gameplay mechanics and content can be implemented during the final milestone.

12.13. Future Roadmap for Milestone 3

With the core gameplay systems now completed, development during Milestone 3 will primarily focus on expanding game content, improving replayability, and polishing the overall user experience.

Our planned objectives include:

- **Era Progression System:** Expand the current game by introducing additional eras, featuring unique unit rosters, upgraded base appearances, and different animations to better capture the progression mechanics inspired by Age of War.
- **Special Abilities:** Integrate cooldown-based global abilities (e.g., calling down a meteor strike on the battlefield).
- **Advanced AI:** Upgrade the EnemySpawner script to read the player's unit composition and intelligently spawn counter-units, rather than relying on randomised timers.
- **Game Balancing:** Perform extensive balancing of unit costs, movement speeds, attack values, health, turret effectiveness, and economy progression to create a fair and enjoyable gameplay experience.
- **Software Engineering Improvements:** Continue refactoring the existing codebase into smaller, more modular components where appropriate, while conducting comprehensive code reviews, additional gameplay testing, and bug fixing to improve maintainability and overall software quality.

By the completion of Milestone 3, we aim to transform the current prototype into a polished, feature-complete game that delivers a cohesive gameplay experience while demonstrating both the technical and software engineering objectives expected of an Apollo 11 Orbital project.

13. Milestone 3: Software Engineering, Gameplay Expansion and Final Polish

Building upon the gameplay systems completed during Milestone 2, the primary objective of Milestone 3 was to improve the overall reliability, maintainability, and gameplay experience of the project. While previous milestones focused on implementing the core mechanics required for the game, this milestone for us centred mainly on refining those mechanics through extensive software testing, architectural improvements, user interface redesigns, and additional gameplay features.

A major emphasis during this milestone was ensuring that newly implemented features could be confidently maintained as the project continued to expand. To achieve this, taking into account the feedback from milestone 2, automated testing was introduced across multiple gameplay systems, requiring significant refactoring of the existing codebase to improve modularity and testability. Beyond software quality improvements, several gameplay systems, including the refactoring of the unit production workflow, meteor strike ability, dynamic era progression, and turret management system, were conducted to more closely replicate the mechanics found in Age of War while providing a smoother user experience.

As such, the following sections describe each major feature developed during Milestone 3, the implementation approach adopted, and the engineering challenges encountered throughout development.

13.1. Automated Testing and Software Quality Assurance

13.1.1. Overview and Thought Process for this feature

As we delved deeper into this project, we started to realise that the number of gameplay system components that we had created had increased significantly. For example, features such as unit spawning, economy management, combat interactions, health systems, and special abilities have become increasingly interconnected. While manually testing each feature after every modification was initially feasible, this approach could become time-consuming and also not really a “best practice” since even small changes would need us to reload the whole game. As for why we always conduct manual testing between even implementations, it is because we would like to find out if even a minor modification could affect other systems that were functioning correctly.

As such, to combat this issue and to adopt a good practice habit, an automated testing framework was introduced using Unity’s Test Framework. Rather than relying solely on manual gameplay testing, automated tests

were developed to verify the correctness of individual systems under controlled conditions. This provided greater confidence that future modifications would not unintentionally break existing functionality.

While carrying out this testing, several architectural limitations were brought to light within the original implementation. This is because for the game we are trying to design, many of our scripts were designed primarily for gameplay functionality rather than software testability, resulting in tightly coupled components and publicly exposed variables that made isolated testing difficult. As a result, a significant portion of this milestone had to be placed into refactoring existing systems to improve encapsulation, modularity, and maintainability while preserving existing gameplay behaviour.

All in all, we feel that the addition of this feature not only improved the reliability of the project but also established a stronger software engineering foundation for future development.

13.1.2. Scripts Created and Modified

To support automated testing, several new test scripts were developed while multiple gameplay scripts were refactored to improve testability.

Scripts Created

- HealthSystemTests.cs
- EconomySystemEditorTests.cs
- EconomySystemPlayModeTests.cs
- PlayerSpawnerPlayModeTests.cs (Has unit tests and integration tests structure as multiple scripts/components are needed to run this test)

Gameplay Scripts Modified

- HealthSystem.cs
- EconomySystem.cs
- PlayerSpawner.cs
- UnitMove.cs

Overall, the created scripts help validate the behaviour of the corresponding gameplay systems under both editor and runtime environments, while the modified scripts include helper methods for obtaining read-only property variables and testing utilities, all to help to set up the environment to verify that

the changes to the variables after running a method/scenario are as desired when creating the function we are testing.

13.1.3. System Functionality (How Unit and Integration Testing Work)

While researching how to set up Unity automatic testing scripts, we found out that Unity provides two primary categories of automated testing, mainly the **Edit Mode Tests** and **Play Mode Tests**.

Edit Mode tests execute directly within the Unity Editor without entering Play Mode. These tests are generally executed quickly, and they are mainly used for validating pure logic that does not depend on Unity's runtime lifecycle (i.e. time passing, object creation and deletion, which need time to pass to check the difference). As such, after doing trial and error and researching the difference between the two modes, we realised that this mode is more tailored to resource calculations, purchasing logic, and mathematical operations.

Play Mode tests, on the other hand, execute while Unity is running the game. These tests simulate actual gameplay behaviour, allowing systems that depend on frame updates, timers, physics, animations, object destruction, or coroutines to be validated under realistic runtime conditions.

As such, to **maximise testing efficiency**, both testing modes were utilised appropriately throughout the project.

For example, the **Economy System** was separated into two distinct groups of tests. Resource spending, purchasing validation, and helper functions were verified using Edit Mode tests, while passive gold generation and passive experience generation, both of which rely on elapsed game time, were validated using Play Mode tests. This separation ensured that each feature was tested within the environment most appropriate for its functionality.

Likewise, for the **Health System**, the tests created ensured that damage calculations, healing, health clamping (For healthbar UI update), and death behaviour functioned correctly under various gameplay scenarios. Since there is a need to check if an object has been destroyed if the unit health point is less than or equal to 0, the Play Mode test environment is the only mode that we can use to test such a case.

Another example would be the **Player Spawner**, which was also tested to verify that units could only be spawned when sufficient resources were available, that cooldowns behaved correctly, and that invalid spawning conditions were handled safely. This was also created in the Play Mode test

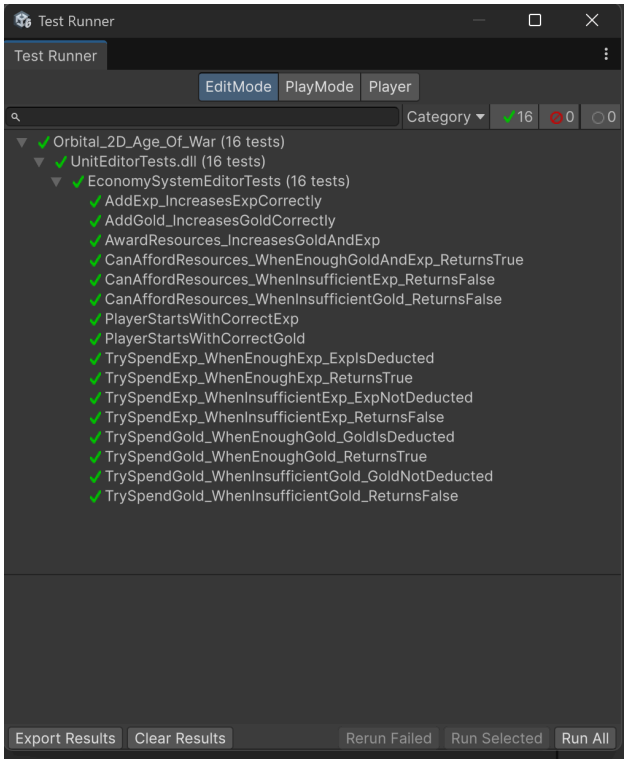
environment since we want to test unit training time cases, where units will only spawn once its training time is completed.

Regarding the scripts that were refactored to improve their suitability for automated testing, public variables that were previously exposed solely for debugging purposes were converted into private fields using Unity's [SerializeField] attribute. At the same time, read-only properties were introduced where external observation was required. Dedicated helper methods were also added to initialise internal states during testing without exposing unnecessary setters during normal gameplay. This was done to ensure that proper immutability properties are upheld in line with good Object-Oriented Programming practices.

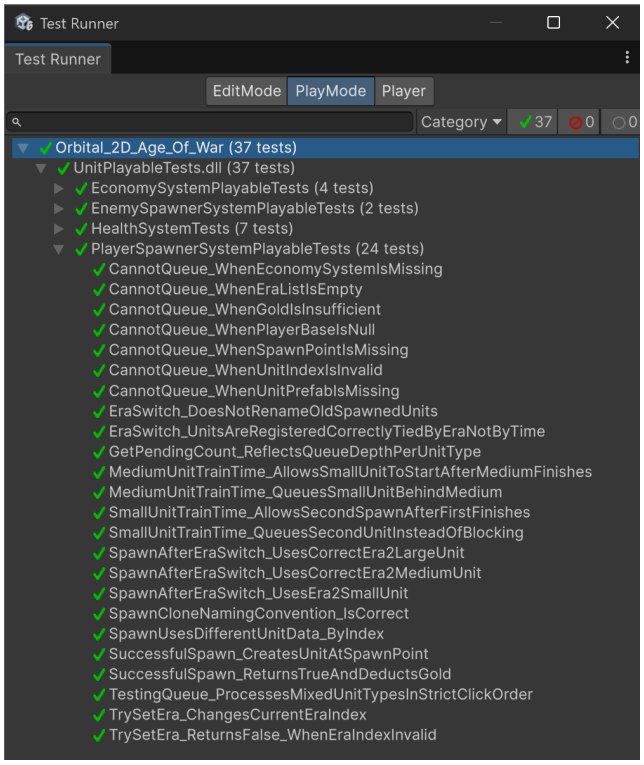
All in all, we feel that the refactoring of these scripts helped to significantly improve the maintainability of the project by reducing coupling between systems while simultaneously making automated testing considerably easier to perform.

The images below show the edit and play mode test script(s) being run to test the method's proper functionality to ensure that the method is working as intended.

Edit Mode:



Play Mode:



13.1.4. Challenges faced during the building process of this feature

1. **Truly Understanding Unity's Testing Framework**

One of the first challenges encountered was understanding the distinction between Unity's Edit Mode and Play Mode testing environments. Initially, I chose to create all automated tests using Edit Mode because, after doing some brief research, Edit Mode usually runs faster when executing the test methods. As such, at a surface level, this appeared to be the most efficient approach, since I felt that the majority of gameplay logic consisted of simple function calls and variable updates.

However, when trying to run the test, several tests repeatedly failed despite the corresponding gameplay functioning correctly during normal execution.

After diving deeper into Unity's Test Runner, I realised that functions like Destroy() do not immediately remove an object when called. Instead, Unity schedules the object for destruction at the end of the current frame. Moreover, since Edit Mode tests do not simulate Unity's runtime frame lifecycle, the destruction request was never processed before the assertions were evaluated. As a result, some test methods incorrectly reported failures even though the implementation itself was correct.

As such, to resolve this issue, the majority of the test scripts that required Unity time states (multiple frames were needed) were migrated into Play Mode, where Unity's normal runtime behaviour could be simulated. By allowing the test to wait for the next frame before performing assertions, testing components like object destruction and unit training time occurred naturally as part of Unity's update cycle.

2. **Refactoring Existing Code for Testability**

Throughout the implementation of this feature, introducing automated testing also exposed several design limitations within our original scripts.

This is because many gameplay scripts were initially developed with rapid feature implementation as the primary objective. As a result, several internal variables were publicly accessible, while certain components depended directly on other gameplay objects existing within the scene. Although this approach was taken to help us in the development of the scripts and concepts, it eventually complicated isolated testing scenarios because individual systems could not easily be instantiated without recreating large portions of the game environment (i.e. a lot of dependencies needed to be defined to test one portion that does not even use some parts that needed to be defined).

Thus, to improve software maintainability, substantial refactoring was carried out across multiple gameplay systems. Internal variables were encapsulated using private fields, while read-only properties were introduced for values that required external observation during testing (usually for data validation to check if calculations are done correctly, instances are stored correctly, etc.). Helper methods were also added to initialise testing environments without exposing unnecessary implementation details to other gameplay systems.

While refactoring required considerable effort, it ultimately resulted in a cleaner software architecture that was easier to understand, easier to extend, and significantly easier to test.

3. **Balancing Testing Coverage with Development Time**

As the number of gameplay systems continued to grow, another challenge I faced was deciding which components should receive automated tests.

While it is possible to attempt complete test coverage across every script, I felt that certain systems, particularly user interface interactions, animation events, and purely visual effects, provided relatively little benefit compared to the development effort required to automate them.

Instead, testing efforts were prioritised towards systems responsible for core gameplay logic, including resource management, unit spawning, combat interactions, and health management. These systems directly influence gameplay correctness and therefore presented the greatest risk of introducing regressions during future development.

This selective approach allowed development time to be utilised more effectively while still achieving high confidence in the correctness of the project's most critical gameplay systems.

13.1.5. Overall Outcome:

The introduction of automated testing represented one of the most significant engineering improvements we made during Milestone 3. Beyond improving software reliability, the process required a substantial redesign of several existing gameplay systems to improve modularity and maintainability.

By distinguishing between Edit Mode and Play Mode testing, refactoring existing scripts for improved encapsulation, and prioritising testing around the project's core gameplay mechanics, we feel that the project now has a solid software foundation compared to the previous milestones. This enables future gameplay features to be implemented with greater confidence while reducing the likelihood of regressions as development continues.

13.2. Queue-Based Unit Production System with UI queue trackers

13.2.1. Overview and Thought Process for this feature

In Milestone 2, the unit deployment system was implemented using a cooldown-based spawning mechanism. This means that whenever the player selected a unit for deployment, the unit was immediately spawned onto the battlefield before entering its respective training cooldown. While this implementation successfully prevented players from repeatedly spawning units (preventing unwanted collision scenarios from happening), we felt that it does not really make much sense in hindsight, since we would like to have a scenario where players are able to continuously queue multiple units. Moreover, we felt that instead of instantly spawning the unit, we tried using the idea of spending gold to invest into a unit to be ready to be deployed, as it feels more realistic.

While looking at our current progress in milestone two, the one thing that was disturbing me was the fact that players were required to repeatedly wait for individual cooldowns to finish before issuing another deployment command,

resulting in unnecessary interruptions during combat. This issue became increasingly apparent during large-scale engagements where players are unable to prepare multiple units in advance, which we felt negatively impacted the gameplay flow.

As a result, to better replicate the original gameplay experience (making it more enjoyable and realistic) and improve responsiveness, the production workflow was completely redesigned as a queue-based training system. Rather than spawning units immediately, deployment requests are now placed into a production queue and trained sequentially before entering the battlefield. This redesign fundamentally changed how the spawning system operated internally while simultaneously introducing several new user interface components to communicate the queue status to the player clearly.

13.2.2. Scripts Created and Modified

The implementation of the queue system required significant modifications to the existing spawning architecture as well as the introduction of several supporting user interface components.

Scripts Created / Modified

- PlayerSpawner.cs (Modified)
- UnitQueueUI.cs (Created)

Supporting UI Components

- Training Progress Bar (To show which unit is currently being trained)
- Queue Count Display (To show the number of units the player selected to be trained and deployed)
- Queue Status Manager (integrated within the spawning system, to help pull data of each unit's training time)

These components help synchronise the production queue to continuously update the player's current training progress and remaining queued units, allowing for better usability and gameplay experience for the player.

13.2.3. System Functionality (How and why did we use Queue Systems)

Unlike the previous implementation (deploy then wait), the unit production is now separated into two distinct stages: queueing and deployment.

Whenever the player selects a unit, the system first validates that sufficient gold is available before deducting the corresponding cost. Instead of spawning the selected unit immediately, a production request is inserted into a First-In-First-Out (FIFO) queue. This guarantees that units are trained in the exact order in which they were requested.

Like how a normal queue data structure works, only the unit positioned at the front of the queue is actively trained. How the flow works is that when a player first clicks on a unit he wants to spawn, it gets added into the queue, after which it checks with a boolean tracker if there is a unit currently being trained. If there is no unit being trained, then we will remove this unit from the queue and start training this unit, in which we will flag the boolean isTraining tracker to be True. Also note that once the unit has been fully trained and has been instantiated at the player spawn point, the next queued request immediately begins training, allowing production to continue automatically until the queue becomes empty.

Overall, we felt that separating production from deployment significantly improves gameplay responsiveness by allowing players to prepare multiple units in advance without repeatedly monitoring individual cooldown timers. This encourages players to focus on battlefield decision-making rather than continuously waiting for production to finish before issuing another command.

Moreover, to complement the redesigned production system, several new user interface elements were also introduced. This unloaded the mental stress placed on the user to constantly memorise which unit(s) have been selected to train and which unit is currently being trained to be deployed, as it is shown directly to them instead of needing to memorise the order the player chose.

To help ease the load for the player, firstly, a dynamic training progress bar was implemented to visually indicate the remaining training time of the unit currently under production. Instead of relying solely on numerical timers, the progress bar provides immediate visual feedback regarding production progress, allowing players to quickly estimate when their next unit will enter the battlefield.

Secondly, queue counters number trackers were added at the top right of each unit icon. These counters display the number of pending production requests for each unit type, enabling players to easily monitor the breakdown of their upcoming army without needing to memorise which units and how many were queued to be deployed.

All together, we feel that these interface improvements provide significantly greater transparency into the production pipeline of the unit(s) training, all while reducing unnecessary player interaction and workload, allowing better user experience for players.

Old Unit UI:

New Updated Unit UI, showing tracking and training queue count:

13.2.4. Challenges faced during the building process of this feature

1. **Redesigning the Original Spawing Architecture (Complete Overhaul)**

The largest challenge encountered during the implementation of the queue system was that the original spawning architecture had not been designed to support queued production. This is because initially, the deployment workflow followed a relatively straightforward sequence:

Player Clicks Unit → Gold Deducted → Unit Spawned Immediately → Training Cooldown Begins

Although this approach was simple to implement, it fundamentally differed from the overall final production behaviour that we were looking for (making it more realistic). As such, to support the queued production, the entire spawning pipeline had to be redesigned (complete overhaul) into the following workflow:

Player Clicks Unit → Gold Deducted → Production Request Added to Queue → Unit Waits in Queue (if there is a unit being trained, else) → Training Begins (in which unit is removed from queue) → Unit Spawned → Next Unit Starts Training.

While this modification in theory appears simple, it required restructuring almost every stage of the existing spawning system. Logic that previously executed immediately after a button press now needed to be deferred until the corresponding production request reached the front of the queue. As a result, almost every single function created for the player spawning concept has to be rewritten, along with adding new methods that help ensure that the queuing concept works as intended, all while ensuring that resource deduction, training progression, and spawning remain correctly synchronised.

Therefore, for this change, rather than simply extending the existing implementation, much of the original spawning workflow was redesigned to accommodate this new production model, which took up a lot of time.

2. **Synchronising Queue gameplay with User Interface (what to show to the Players)**

Once queued production had been implemented successfully, another challenge emerged, which was to know what variables were needed to communicate the queue state to the player.

While attempting to fix this problem, we felt that simply displaying the current unit that is being trained was not enough, as players also needed to know how many additional units remained queued behind it. Furthermore, we felt that the players needed to know which unit is currently being trained, as well as how many pending units for each unit type have been selected to be trained by the player.

Initially, queue information was accessed directly by multiple interface components. However, this approach quickly became difficult to maintain as several interface elements required the same production data. This can become very messy very quickly as any changes on one component may not be reflected correctly on the other components. As such to address this issue, a centralised manager was introduced to coordinate queue information before

updating the relevant interface components. Instead of allowing every interface element to inspect the production queue independently, the manager retrieved the required production state before distributing simplified information to the training progress bar and queue counters.

By taking on this approach, we helped reduce coupling between gameplay logic and presentation, making both systems easier to maintain while ensuring that all user interface components remained synchronised with the current production state. As such, this means that the training bar as well as the unit count in the queue is reflected correctly for each unit that can be deployed right now, without needing to worry about synchronisation issues.

13.2.5. Overall Outcome:

Overall, we feel that the queue based production system represents one of the largest architectural improvements introduced during Milestone 3. By redesigning the spawning workflow from an immediate deployment model into a sequential production pipeline, the gameplay experience now feels more realistic while significantly improving player responsiveness.

Beyond the gameplay improvements, implementing the queue system also required substantial refactoring of the spawning architecture and user interface, really reinforcing the importance of modular software design as projects continue to grow in complexity (a pain point that we had to go through).

13.3. Dynamic Era Progression System

13.3.1. Overview and Thought Process for this feature

One of the defining mechanics of Age of War is the player's ability to evolve through multiple technological eras, unlocking progressively stronger units, defensive structures, and abilities as the game progresses. During the early stages of development, unit data and user interface elements were largely hardcoded around a single era, specifically utilising folder scanning and index based onClick() delegates. While this approach was sufficient for initial prototyping, we realise our current approach would not be able to handle game scalability in the future.

As a result, when additional eras were being introduced into the game, the original implementation of loading player units by directly scanning prefab folders became increasingly difficult to maintain (how do we tag units names such that one button can be used for multiple units, as we did it by number

based onClick() method calls originally). This approach resulted in every new era requiring manual updates across multiple gameplay and user interface components, resulting in duplicated logic and making future expansion cumbersome.

Therefore, to improve scalability, the project was redesigned to use data-driven architecture. Instead of treating each era as a collection of unrelated prefabs, a dedicated data class was introduced to encapsulate all information associated with a single era. This includes the era name, playable units, user interface artwork, evolution requirements, and special ability configuration. By grouping related information together, the gameplay systems only need to reference the currently active era, greatly simplifying both implementation and future expansion. (Overall utilising new data classes and array data structure to our advantage)

13.3.2. Scripts Created and Modified

Scripts Created / Modified

- PlayerSpawner.cs (Modified)
  - Manages the player's current era.
  - Retrieves the unit informations form the active era (unit cost, train cost)
  - Refreshes gameplay and user interface elements after evolution.
- TurretManager.cs (Created)
  - Updates the available turrets based on the player's current era.
  - Refreshed turret cost and toolbar artwork according to the era we are in.

Data Classes Introduced (for storing necessary data by era)

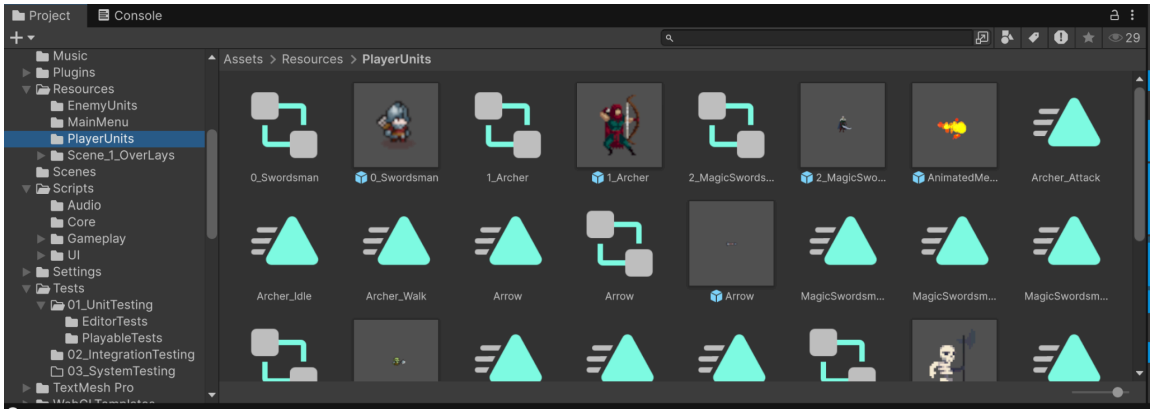
- **PlayerEraData**
  - Stores all player related data belonging to a particular era, such as:
    - Era name
    - Playable units
    - Evolution EXP cost
    - Special Ability (Meteor Strike) configuration [Number of meteors, cost for ability in Exp)
    - Unit banner artwork (For UI image changes when era changes)
- **TurretEraData**
  - Stores all turret related data belonging to an era, such as:
    - Era name
    - Number of turrets for each era

- Turret Names
- Turret costs (Gold)
- Turret toolbar artwork (For UI image changes when era changes)

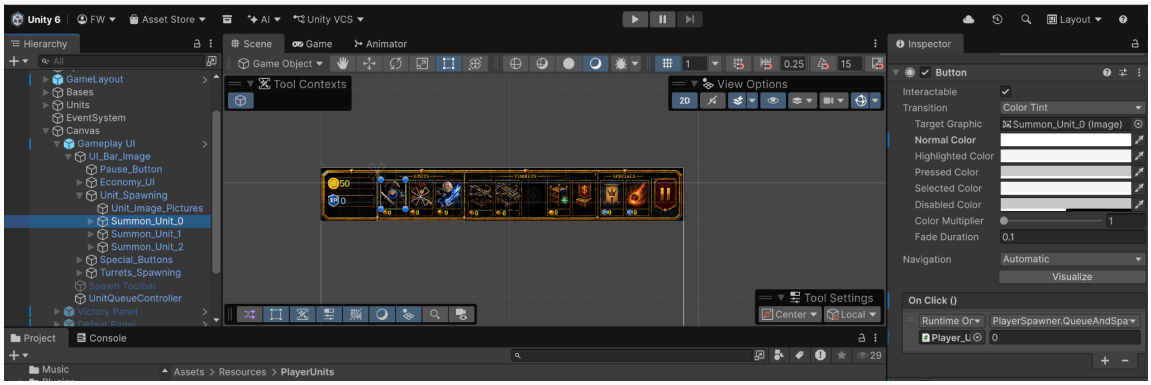
By introducing these intermediate data classes, gameplay systems no longer need to know the details of every individual era, allowing them to retrieve all required information from a single object, making it cleaner and easier to use.

Before Changes:

Unit Prefab Folder:

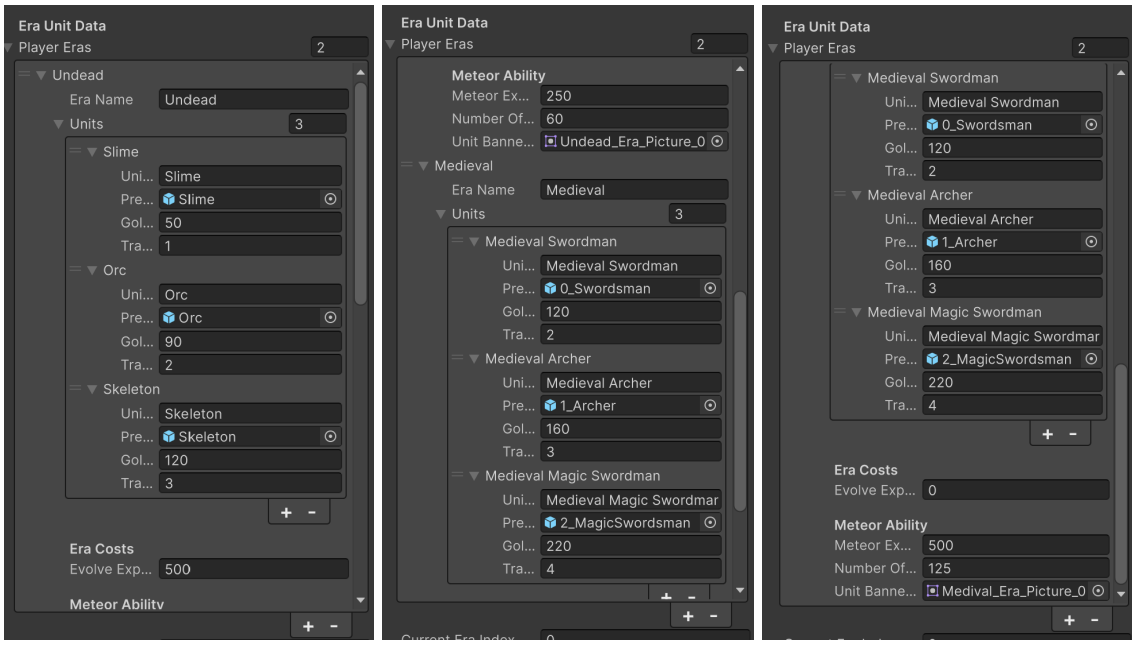


Unit Button spawning Configurations:

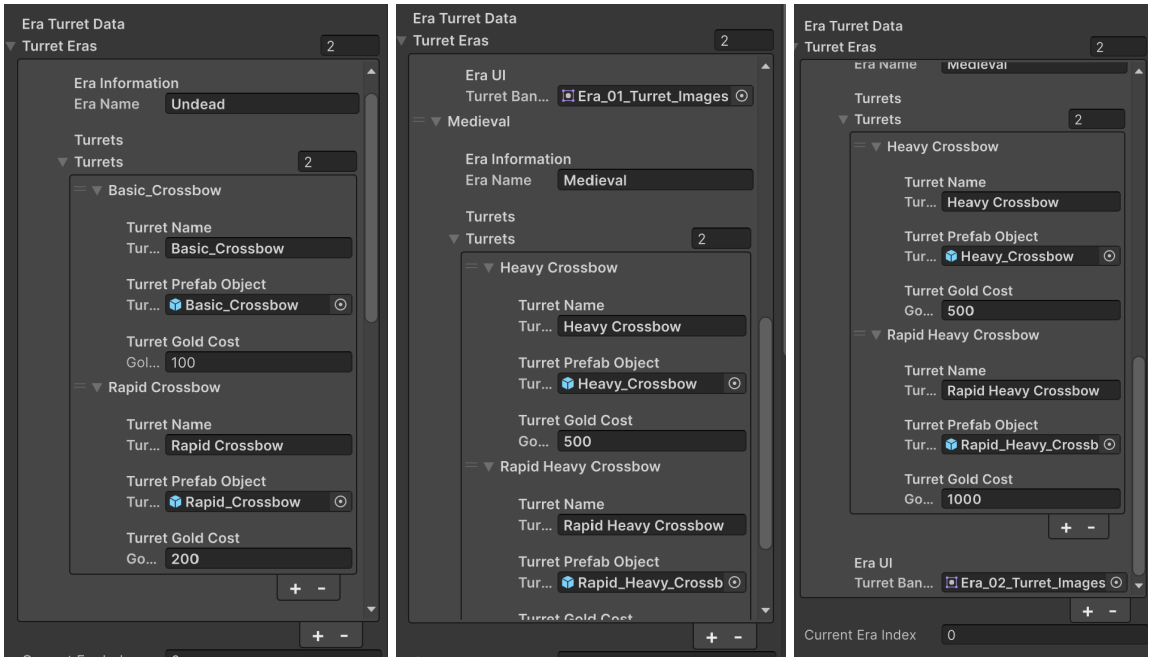


In the images above, our original setup was for each individual button to have an onClick action that would give in a specific value, in which we scrape this folder to find the unit that matches the value. In this case, 0 → Swordman, 1 → Archer, 2 → Magic Swordman. Through this set up we realise that to expand the set up for multiple eras, we essentially need to duplicate this entire set up to help consider other eras starting from 3, 4, 5 so on and so forth. Before blindly brute forcing this solution, we felt that we should just try and future proof the idea by making it more scalable.

After Changes:



In the images above, we instead use the data classes we introduced to store all the necessary data needed for every era units, make it more developer friendly as now instead of constantly needing to add new methods each time we want to expand the eras further, we can now Player Eras array list to store more era with each era “troops” details, saving future time and less complicated for the users expand the game further. The same idea is also used for different turrets for different eras, as shown in the images below:



13.3.3. System Functionality (How we implement the Era Progressions)

As shown above, each era is represented by a dedicated data object containing both gameplay (unit and turret prefab, which contains all of the unit data) and presentation (Unit and Turret banners) information. Rather than individually updating units, costs, banners, and special ability values, the PlayerSpawner only needs to retrieve the currently active PlayerEraData. Similarly, the TurretManager retrieves the corresponding TurretEraData for the same era.

This would mean that whenever the player evolves, only the current era index is updated. In which the relevant gameplay systems will refresh themselves using the newly selected era data. This automatically updates:

- Available player units
- Unit deployment costs
- Unit banner artwork (for each era)
- Evolution EXP cost (if there is a next era, else it will show MAX)
- Special Ability (Meteor Strike) configuration (numbers of meteors, cost of ability)
- Available turret to be deployed
- Turret toolbar artwork (for each era)

As mentioned earlier, since every configurable value is stored inside the era data classes, adding a new era requires little more than creating another data object (Player/Turret data, i.e. expanding the array list) and assigning the corresponding assets within the Unity Inspector. Even after the expansions, existing gameplay logic continues functioning without requiring additional conditional statements or hardcoded values, making it scalable for the future.

Undead Era UI bar (Era 1):

Medieval Era UI bar (Era 2):

The images above show the UI changes based using the new data class concept to store unit and turret information, as well as its banner images.

13.3.4. Challenges faced during the building process of this feature

1. **Eliminating Hardcoded Era logic**

As mentioned earlier, the previous implementation relied directly on referencing player prefabs when spawning units. While this approach was suitable during the early stages of development, supporting multiple eras would have required numerous conditional statements to determine which units, costs, and artwork should be displayed.

Therefore, to improve maintainability and scalability, an intermediate data layer was introduced through PlayerEraData and TurretEraData. Instead of querying multiple variables individually, gameplay systems now retrieve all relevant information from the currently active era object. This significantly reduced duplicated logic while making the system considerably easier to extend.

This was somewhat tedious as during this milestone, I had to refine the data class multiple times as I forgot some attributes that are needed in the era (special ability data), which while not too taxing, was quite hard to get it right in one go.

2. **Synchronising Gameplay and User Interface**

While trying out the changing of eras, we realise it is not as simple as just changing the playable units/turrets that can be spawned when the era changes. Unit costs, toolbar artwork, meteor ability configuration, evolution costs, and turret information must all transition simultaneously to avoid presenting inconsistent information to the player.

Initially, these interface elements were updated independently, making it easy to overlook one during development. To address this, dedicated refresh methods were introduced to centralise all era-related updates (In this case we put these methods in the Player Spawner as it contains almost all of the data needed in the UI taskbar).

As a result, whenever the active era changes, the gameplay systems automatically refresh every dependent interface component using the data stored within the new era object. This centralised update process ensures that gameplay behaviour and user interface presentation remain synchronised throughout the evolution process.

13.3.5. Overall Outcome:

By introducing dedicated era data classes, the progression system evolved from a hardcoded implementation into a scalable, data-driven architecture. This means that gameplay systems are now largely independent of individual eras, requiring only a reference to the currently active data object to obtain the necessary configuration.

Besides simplifying future expansion, this design also improves code readability by grouping logically related information together, allowing gameplay behaviour and user interface updates to remain consistent while significantly reducing duplicated code.

13.4. Introduction of Special Ability (Meteor Strike)

13.4.1. Overview and Thought Process for this feature

To introduce greater strategic depth into gameplay, a special ability system was implemented during Milestone 3. The ability added in this milestone was meteor strike, an area-of-effect attack capable of damaging multiple enemy units simultaneously. Unlike normal unit production, which provides a “continuous” (if we have the resources to generate it) stream of reinforcements, the meteor strike ability serves as a powerful tactical ability that allows players to temporarily regain battlefield control during overwhelming enemy pushes.

Rather than functioning as a conventional projectile, the ability creates a meteor shower across the whole battlefield where the units walk. This encourages players to carefully manage their experience points, as activating the ability requires sacrificing resources that could otherwise be used for progressing to the next era. We feel that using exp as the cost for special abilities increases the complexity of the game, requiring the players to be more resourceful.

13.4.2. Scripts Created and Modified

Scripts Created

- MeteorStrikeAbility.cs
  - Controls the spawning of meteors (spawn range and height, being set up as PlayerBase <------- Spawn range ----------> EnemyBase)
  - Configures the number of meteors and EXP cost for each era.
  - Randomises spawn positions within the battlefield.

- MeteorProjectile.cs
  - To make the Meteor Shower ability feel more dynamic and strategic, this script causes meteors to descend at an angle instead of falling vertically. This creates a more realistic meteor shower effect while increasing gameplay complexity, as players must carefully time and position their ability usage. Since the meteors require time to travel across the battlefield, casting the ability too late may cause fast-moving enemy units near the player's base to evade the impact. This encourages players to anticipate enemy movement and use the ability proactively rather than relying on it as an instant screen-clearing attack.

Scripts Modified

- PlayerSpawner.cs (As mentioned earlier)
  - Updates the Meteor Strike configuration whenever the player evolves.
  - Dynamically refreshes the ability's EXP cost using the current era's data.

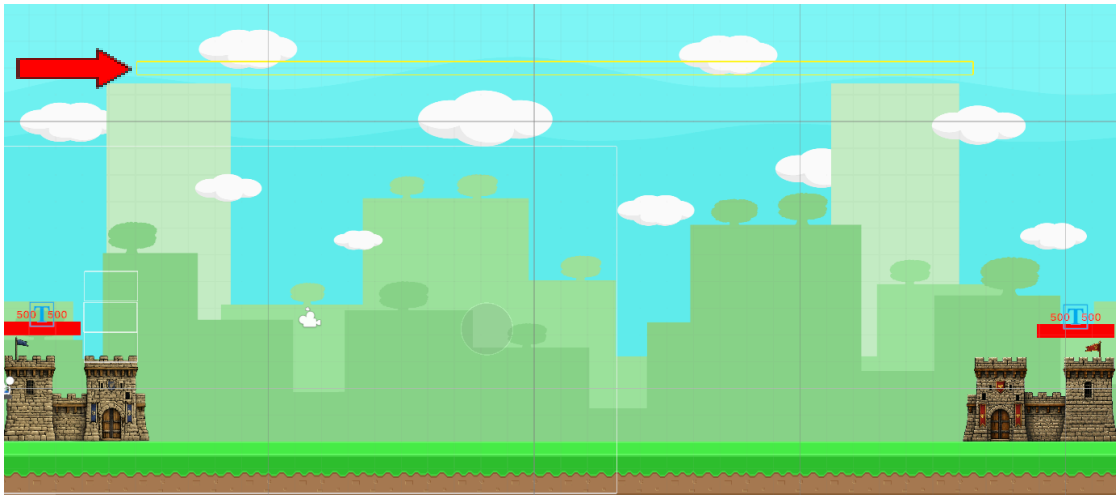
13.4.3. System Functionality (How we implement the Meteor Shower Ability)

For the special ability, Meteor Shower, it is activated by clicking on the button with the meteor shower icon. Upon activation, meteors are spawned sequentially at random positions between the two bases from the sky, falling diagonally towards the ground. Once a meteor collides with the ground layer or enemy unit, it creates an explosion that damages nearby enemy units within a predefined radius. Note that in the gameplay, we set it up such that only the enemy unit will get affected by the meteor shower. This is because allowing the enemy base to get damage from the meteor shower can make the gameplay too easy, as players just need to constantly use the ability to win.

By allowing the configuration of the special ability, i.e. the meteor shower, to be specified in the PlayerEraData (via the inspector UI), we allow future eras to easily introduce stronger or weaker versions of the ability without needing to modify the underlying gameplay logic.

As mentioned previously, the Meteor Shower is configured to spawn only within the battlefield between the two bases. Since enemy units traverse only this section of the map, restricting the spawn area ensures that meteors remain within the playable area while maximising the likelihood of striking enemy units. This also prevents meteors from falling outside the map boundaries and reinforces the intended design that the ability is used to

eliminate enemy units rather than directly damaging the enemy base. The image below reinforces the idea as the yellow box shown in the sky is the initial spawn area of the meteors, which spawn at random positions in the box.



13.4.4. Challenges faced during the building process of this feature

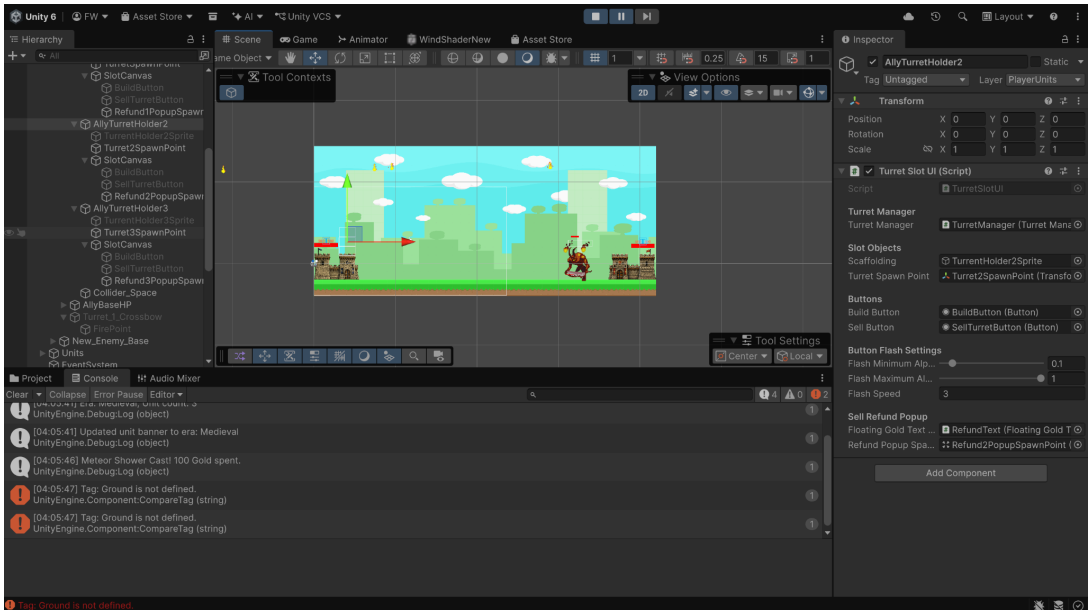
1. **Restricting Meteors to the Battlefield**

As shown above, one of the first issues encountered was determining where meteors should spawn. Initially, random positions were generated without considering the playable battlefield, causing meteors to occasionally land outside the intended combat area. This reduced the effectiveness of the ability while also creating inconsistent gameplay.

To resolve this issue, meteor spawn positions were constrained to the enemy side of the battlefield, ensuring that every meteor contributed meaningfully during combat. (Image shown above)

2. **Detecting Ground Collisions**

Another challenge involved determining when a meteor should explode. Our first attempt at this was to reference specific ground objects directly. However, because prefabs cannot maintain references to scene objects, this approach was unreliable and difficult to maintain. Below shows the image of the error shown during the runtime / play testing, the meteors are unable to find the ground correctly, thus throwing an error and “crashing” the game.



Instead, we chose to do collision detection though the usage of layers, like the Ground layer rather than a specific GameObject. This allowed every meteor prefab to detect impact correctly regardless of the scene in which it was instantiated.

3. Preventing Damage to the Enemy Base

For our first attempt, meteor explosions damaged every object possessing a health component and enemy tag, which includes the enemy base. As a result, from play testing, players could repeatedly cast Meteor Strike to destroy the base without engaging in normal combat, significantly reducing the strategic importance of units.

To preserve gameplay balance, the explosion logic was modified to damage only enemy units. Since all combat units contain both HealthSystem and UnitMove components scripts while the enemy base does not, the ability now ignores structures (those without UnitMove components) automatically without requiring additional tags or special-case checks.

13.4.5. Overall Outcome:

The Meteor Strike ability introduces an additional layer of tactical decision making by allowing players to exchange experience points for powerful battlefield control (making them weigh the cost of upgrading to a new era or using the special ability). Through configurable era based parameters and improved collision handling, the system remains flexible and easy to use while integrating seamlessly with the existing progression architecture.

13.5. Overhauling the Dynamic Gameplay User Interface [UI] (Action Bar)

13.5.1. Overview and Thought Process for this feature

As new gameplay systems were introduced throughout Milestone 3, the existing user interface became increasingly insufficient as attempting to merge new components (to showcase the added gameplay information) with the current UI was challenging. Features such as unit production queues, era progression, special abilities, and turret management all required the interface to update dynamically based on the player's current game state.

Therefore, to improve both usability and scalability, the gameplay interface was completely overhauled and redesigned so that all displayed information is generated dynamically from the underlying gameplay systems rather than remaining statically configured within the Unity Editor, all while ensuring that the theme between all the UI components mesh well together .

13.5.2. Scripts Created and Modified

Scripts Created

- PlayerSpawner.cs (Modified)
  - Updates each unit costs (Based on the era we are in).
  - Refreshes unit banner artwork (So that the unit art work matches what is being spawned).
  - Refreshed Meteor Strike (where further era, the meteor strike is more powerful, but cost more as well).

- TurretManager.cs (Created)
  - Updates each turret costs (Based on the era we are in).
  - Updates turret artwork (So that the unit art work matches what is being spawned).
  - Control button availability based on the current era, as there is an empty slot for turrets for certain eras, thus we need to control if the button is clickable or not.

UI Components Added / Changed

- Refractor the Gold and Exp tracker.
- Refractor the Unit spawn banner and made it dynamic (refined the unit images and shifted the unit cost text for each unit).
- Added a dynamic turret banner (includes turret Image, turret cost, turret holder and sell buttons as well).

- Added a special ability section (includes the evolution but that show the evolution cost in exp, as well as meteor special ability, showing the meteor exp cost as well).
- Added training progress bar for each unit, where it only showcases when the unit is the one that is currently being trained.
- Added units to be trained counters so that the player know what units has / have been selected by them to be trained.

13.5.3. System Functionality (How and why we overhauled the Action bar)

Rather than manually creating a separate action bar for every era and switching between duplicate UI GameObjects whenever the player evolves, the interface we created retrieves its information directly from the currently active gameplay data (Mainly from the script data mentioned above), updating the necessary images and text at one go.

As a result, this means that whenever the player evolves, the PlayerSpawner automatically refreshes the displayed unit banner, unit costs, evolution cost, and Meteor Strike information using the active PlayerEraData. Likewise, the TurretManager updates the available turret artwork and corresponding costs using the current TurretEraData. By setting it up to be dynamic, instead of needing multiple game objects, or updating each component button individually, we can just update a picture of all the unit(s) / turret(s), as well as the cost, at one go, making it cleaner and easier to maintain.

Moreover, additional interface elements were also introduced to support the queue based production system. A training progress bar continuously displays the progress of the unit currently undergoing training, while unit counters (+1, +2, etc) indicate the number of pending production requests for each unit type.

By deriving interface information directly from gameplay data, the displayed information always remains synchronised with the underlying game state without requiring manual updates.

As for the choice of the redesign of the UI, and how we managed to obtain the UI redesign, since our original UI had the following layout:

Gold counter EXP counter	Blank Space	Pause Menu	Blank Space	Unit Action Bar

However, as additional gameplay features, such as turret selection and special abilities, were introduced, the original user interface became increasingly cluttered. Moreover, simply placing these new components into the remaining

empty spaces resulted in an inconsistent layout, where related gameplay actions were separated across different parts of the screen. This reduced the logical flow of information and made the interface less intuitive for players to navigate.

Furthermore, the interface lacked a consistent visual theme. The resource counters, action buttons and newly added controls adopted different colour schematics and design styles, resulting in an inconsistent appearance. From left to right, the interface transitioned between a prominent gold border, to wooden shield and then silver and beige elements, causing the action bar to feel like several unrelated components rather than a cohesive user interface.

Image of the Old Action Bar:

As a result, the action bar was redesigned to provide a consistent visual theme while maintaining the overall medieval warfare aesthetic of the game. The interface was reorganised into a single horizontal action bar, with gameplay elements arranged according to their frequency and logical order of use: resources (Gold and EXP), followed by units, turrets, special abilities, and finally the pause menu. This grouping ensures that related actions are presented together, improving both usability and readability while creating a more cohesive interface. As for how we managed to obtain this action bar, ChatGPT was used to generate a unified action bar concept image, which was then incorporated and adapted into the final game UI.

Image of the new and Improved Action Bar:

13.5.4. Challenges faced during the building process of this feature

1. **Ensuring Consistent User Interface States**

Currently, the gameplay we are aiming to create contains actions that can simultaneously affect multiple interface components. For example, when a player wants to evolve to the next era, changes in the unit artwork, unit costs,

turret information, special ability costs, and evolution requirements should be updated at the same time.

Initially, our interface elements were configured to be refreshed independently. As a result, if not configured properly, certain interface elements could be missed out or become outdated, causing incorrect information to be displayed to the player.

Therefore, to prevent such scenarios from happening, we grouped related interface updates into dedicated refresh methods, ensuring that every affected interface component is updated together whenever the underlying gameplay state changes.

2. **Ensuring AI image generation matches the theme for updated era units**

While AI was able to generate me an action bar banner that matched my needs for the baseline, there was quite a challenge to request for a separate section to AI art that matches what was created, only changing the Unit/turret image. This took up several hours of constantly waiting for it to generate, only for image corruption, images becoming blurry, generating the totally wrong concept.

Throughout this entire process, I realise that AI does better with proper references, as well as not giving them images that are too cluttered, as they consider the external factors as part of the template, even when specified to ignore them.

13.5.5. Overall Outcome:

The redesign of the user interface was ultimately necessary as it significantly improves both gameplay readability and software maintainability. By deriving all displayed information directly from the underlying gameplay systems (as mentioned earlier how and why it was configured as such), the interface is able to remain synchronised with the current game state at all times, all while reducing duplicated update logic. This architecture setup also provides a scalable foundation for future gameplay features and additional eras.

13.6. Dynamic Turret Management System (Upgrade the UI as well)

13.6.1. Overview and Thought Process for this feature

During Milestone 2, turret interactions were intentionally kept simple to validate the core gameplay mechanics. Turret controls were positioned directly above the player's castle, requiring players to interact with individual turret slots to determine both the turret holder and the turret build position. While functional, this interface became increasingly cluttered as additional gameplay features were introduced and did not scale well with multiple turret types or future eras.

Therefore, to improve usability, all turret related actions were migrated into the main gameplay toolbar during Milestone 3. The construction flow was also changed. Instead of selecting a physical slot before choosing a turret, the player now selects the desired turret first, only if there is an empty turret holder, before choosing from the valid placement locations where to place the turret selected. Additional features such as sequential holder upgrades by clicking one button instead of multiple buttons at each level, turret selling, interaction modes, flashing overlays, cancellation controls and refund feedback were also introduced.

13.6.2. Scripts Created and Modified

Scripts Created

- TurretSlotUI.cs (Modified)
  - Represents an individual physical turret holder.
  - Controls holder unlocking and turret placement.
  - Displays contextual Build and Sell icons.
  - Produces visual flashing effects for available actions (build / sell).
  - Tracks the original turret purchase cost for refund calculation.
  - Upon selling the turret in the slot, gold text showing how much money was earned back is shown.

- TurretManager.cs (Created)
  - Controls all turret related gameplay interactions.
  - Manages Normal, Build and Sell modes (build and sell modes have a unique Overlay).
  - Handles sequential turret holder purchases.
  - Coordinates turret selection, construction and selling.
  - Updates turret information dynamically when the player evolves.
  - Controls the availability of turret toolbar actions (if the buttons can be clicked / can be seen or not).

- FloatingGoldText.cs (Created)
    - Displays the gold returned after selling a turret.
    - Moves the refund text upwards before gradually fading out.
    - Uses unscaled time so that the animation can complete even when gameplay is paused (which makes sense since the turret has already been sold).

Supporting Data Classes

- TurretData
    - Stores the turret prefab, name and construction cost.

- TurretEraData
    - Stores all available turret information for an era, including toolbar artwork and all turret data for each eras.

13.6.3. System Functionality (How and why we updated the turret gameplay

One of the main changes introduced during this Milestone 3 was the redesign of the turret interaction workflow. Previously, turret related controls were positioned directly above the player's castle. In the redesigned version, all turret actions are presented within the main gameplay toolbar (action bar). The player first selects a turret from the available catalogue (if they have enough resources and a turret slot is available), causing the system to enter **Build Mode**. In build mode, only unlocked and empty turret holders then display their Build buttons, allowing the player to choose the desired placement location.

Turret holders are purchased sequentially rather than being immediately available. Each successful purchase unlocks the next holder in the configured slot order, while later holders can be configured to require additional gold (which was what we had done). This produces a more structured defensive progression and prevents the player from immediately accessing every turret position.

A new selling system was also introduced. Selecting the Sell action button causes the system to enter **Sell Mode**, after which only holders containing an existing turret display a Sell button. Selecting one of these holders destroys the turret, returns 50% of its original construction cost and makes the holder available for a replacement turret. The percentage value to be returned could be set based on what the user wants, as the refund method takes in a percentage value of the developer choosing if they want to make the game easier or more punishing for bad plays made by the player.

Another thing to note is that during the Build and Sell modes, the texts that showcases the locations of the turrets to be built or the turrets to be sold will flash, ensuring that it catches the player attention that these are the locations available to execute their desired actions, with the flashing helping them see the actually location behind the text when the text disappears for a moment.

Overall, I have configured the system such that it operates using three mutually exclusive interaction modes:

Normal Mode (None) → regular gameplay with no active turret action.

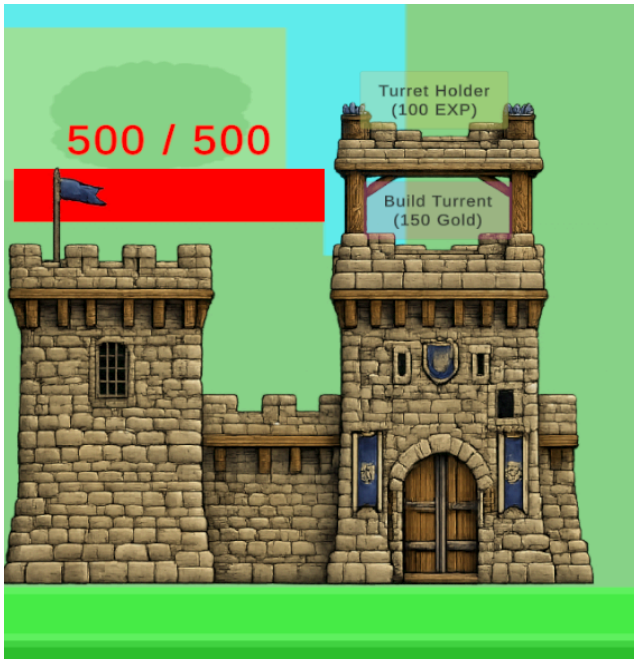
Build Mode (Build) → valid empty holders display flashing Build buttons.

Sell Mode (Sell) → occupied holders display flashing Sell buttons.

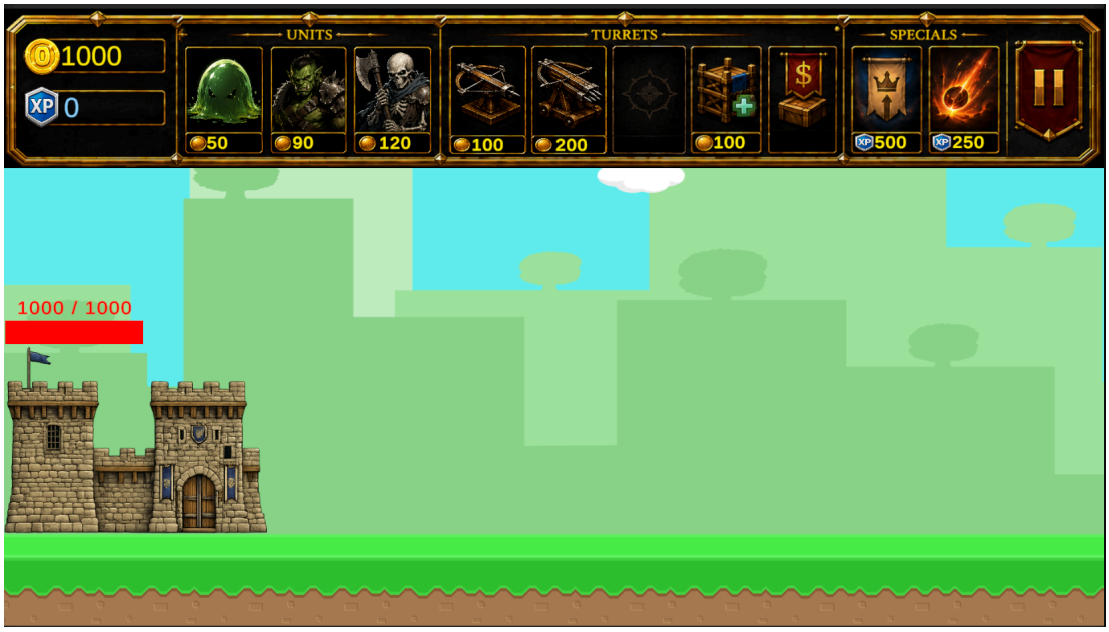
Whenever the player enters Build or Sell Mode, a large Cancel button is displayed in the turret toolbar. This gives the player a clear method of abandoning the current action and returning to Normal Mode.

Lastly, to communicate successful selling, a temporary floating gold value is displayed near the relevant turret slot. For example, selling a turret may display +100 before the text moves upwards, fades out and destroys itself. This provides immediate confirmation of the transaction without permanently occupying interface space.

Old System (where the buttons are directly above the castle):



New Turret System UI (Normal Mode):



New Turret System UI (Build Mode):

Note: As shown in the image, the build button is currently being displayed where it is at a translucent state since the clickable was in the middle of flashing.

New Turret System UI (Sell Mode):

Note: from the two images, it can be seen the selling button clickables are being shown only on turret holders with turrets currently being deployed there, whereby the button flashes so that the players know which button is currently being shown, to ensure they know which turret they are selling.

13.6.4. Challenges faced during the building process of this feature

1. **Managing Multiple Interaction Modes**

In this milestone, while trying to merge the buying and building of turrets into the UI toolbar (Action bar), including the addition of a sell mode to destroy the turrets created and earning a refined, we realised that the current implementation in Milestone 2 is simply not enough.

Therefore, to solve this issue, we introduced a centralised state control, called TurretManager, where we defined and utilized mode tracking of the turret states, mainly the three explicit modes: Normal, Build and Sell. Each mode determines which slot buttons are visible and which actions are currently accepted. Completing an action or pressing the Cancel button resets the system to Normal Mode.

By introducing this concept, we help to reduce conflicting inputs and make the turret workflow easier to reason about and test.

2. **Making Valid Actions Clearly Visible**

During early testing, simply making valid turret holders clickable did not provide enough visual guidance. This is because players could overlook which holders were available for construction or which existing turrets could be sold.

Therefore, to improve clarity valid Build and Sell buttons were configured to flash by repeatedly changing their transparency. We configured the animation to use a sine based value to smoothly alternate between the configured minimum and maximum opacity. This draws the player's attention towards the available actions without requiring additional instructional text.

As previously mentioned, the buttons were also designed to remain translucent rather than fully opaque. This allows the player to still see the turret or holder positioned behind the overlay. In Sell Mode, this is particularly important because the player must be able to identify the turret they are about to remove.

3. **Making Valid Actions Clearly Visible**

Another issue that we had faced was overlay issues, as we configured the gameplay such that turrets would only be instantiated after the scene has already started. Depending on their sorting configuration, a newly spawned turret could appear in front of the Build or Sell button, partially or completely hiding the intended interaction overlay.

To resolve this, the sorting layer and order of the slot overlays were configured to remain above the maximum layer used by any turret sprite. This guarantees

that the Build and Sell controls remain visible and clickable regardless of which turret prefab is instantiated beneath them.

This also preserved the desired translucent effect, where the overlay remains clearly visible while the underlying turret can still be recognised (mainly at moments where the buttons overlay is the most translucent).

4. Communicating the Selling Refund

When implementing turret selling, an important user-interface decision involved had to be made, which was determining how the refund should be communicated.

One option considered was displaying a permanent refund indicator such as a half-coin symbol or 0.5× value on the Sell button. However, upon further consideration, this would have added more visual information to an already compact toolbar and would not clearly communicate the exact amount returned for turrets with different construction costs.

Instead, as we configured the turretSlotUI scripts to contain the original purchase cost of the turret built in that slot, using the stored value to calculate a 50% refund when the turret is sold. As such, since we already managed to configure and have the exact amount that we will be refunding the player, we decided that a temporary floating gold message that is displayed near the affected holder would be a better approach. For this message, it would be a text that rises upwards and gradually fades before being destroyed.

This solution was overall selected and implemented because it provides immediate and precise feedback while keeping the toolbar uncluttered. It also gives the sale a more noticeable visual response than silently increasing the player’s gold total. The image below shows the floating gold text refund in action.

13.6.5. Overall Outcome:

The redesigned turret management system transformed the previous castle based button interface into a more polished and scalable toolbar workflow. Players can now unlock holders sequentially, choose a turret before selecting its placement location, sell existing turrets for a partial refund, and cancel active actions at any time.

Moreover, the addition of explicit interaction modes, flashing translucent overlays and temporary refund text also improves user experience significantly. Through these changes, the turret system now communicates available actions more clearly while remaining flexible enough to support additional turret types, eras and balance changes in future development.

13.7. Gameplay Balancing and Fine Tuning

13.7.1. Overview and Thought Process for this feature

After completing the main gameplay systems, multiple rounds of manual playtesting were conducted to improve combat balance, resource progression and overall match pacing.

Although each mechanic functioned correctly independently, their interaction occasionally created unintended strategies. For example, players had to rely heavily on turrets to defend the base, since the unit's interactions were not favourable to the players, accumulating resources over time, and later use

repeated Meteor Strikes to clear enemy waves before deploying a large group of units to push through to the enemy base without much resistance.

Therefore, to ensure smooth gameplay, gameplay values were therefore treated as adjustable parameters rather than fixed constants. Unit statistics, enemy rewards, turret costs, special ability costs and spawn timings were repeatedly modified according to observed gameplay outcomes.

The objective was not to make every unit equally powerful. Instead, each unit was assigned a clear role within the overall combat hierarchy while ensuring that multiple gameplay strategies remained viable.

13.7.2. Balancing Methodology

Balancing was performed through an iterative playtesting process:

1. Complete a gameplay session using a particular strategy.
2. Identify units or mechanics that appear disproportionately strong or weak.
3. Adjust the relevant gameplay values.
4. Repeat the playtest and compare the outcome.

During the testing process, the main areas that were being evaluated during the playtest were:

- Player and enemy unit matchups
- Gold and EXP progression
- Turret effectiveness
- Meteor Strike usage
- Enemy spawn timings
- Base survivability
- Progression between eras
- Overall match duration

This process checking allowed balancing decisions to be based on actual gameplay behaviour rather than theoretical values alone.

13.7.3. Unit Combat Relationships

In regards to unit interactions, we tried to do player and enemy unit balancing around these combat relationship ideas, which are:

Unit Relationship	Intended outcome
Player Slime ≤ Enemy Fire Mage ≤ Player Swordman	The Slime would be weaker than the Fire Mage, while the Swordman in a one on one battle should be able to defeat the fire mage reliably.
Player Orc ≤ Enemy Archer ≤ Player Archer	The Player Archer should outperform the enemy range unit in a one or one scenario, while ensuring that the orc does not get completely destroyed by the enemy archer before landing a hit.
Player Skeleton < Enemy Golem ≤ Player Magic Swordman	The Skeleton should remain weaker than the Golem (logically speaking), while the Magic Swordsman can defeat the Golem.

In order to incorporate these relationships ideas, following the general strength hierarchy defined above, while preserving different combat roles, the balancing process considerations are for each player unit, their:

- Maximum health
- Attack damage
- Attack cooldown
- Movement speed
- Attack range
- Gold cost
- Training duration

For enemy units, gold and EXP rewards were also balanced according to the difficulty of defeating them. We decided to introduce gold and EXP units for each enemy unit following suggestions from Milestone 2 evaluation, where we decided to shift from passive income generation to active gameplay income generation. Note that the feature of passive income generation is not deleted, but rather unchecked, such that if the user wants to allow both passive and active income generation, to make the gameplayer easier, he/she is able to do so by checking the button in the Economy Manager.

13.7.4. Economy and Reward Balancing

Expanding on the passive and active income generation idea, we felt that one of the main balancing challenges was determining how the player should earn resources.

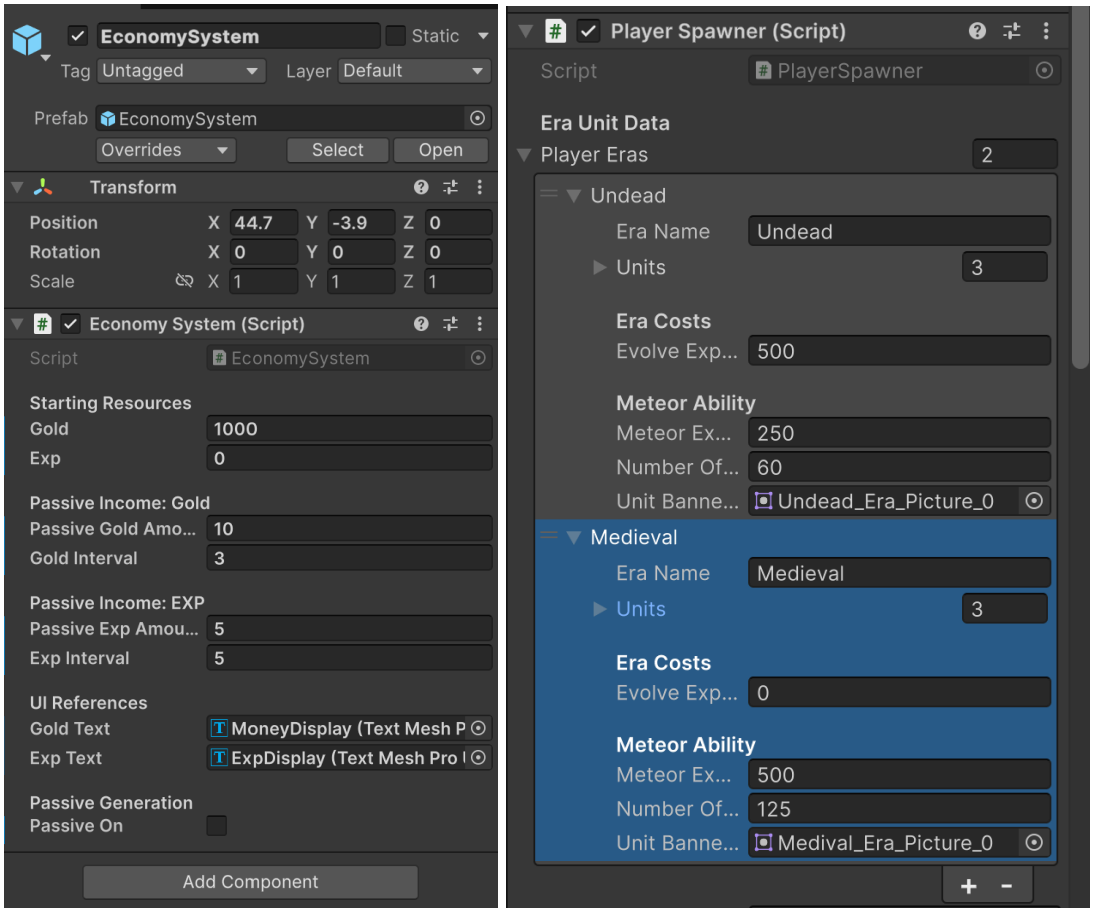
Earlier versions included passive gold and EXP generation. However, upon further deliberation, we felt that this encouraged the player to remain behind the base and wait rather than participate actively in combat. By delaying unit production, the player could eventually accumulate enough resources to construct several turrets and deploy a large army without defeating many enemies.

Thus, in this Milestone 3, we decided to deactivate passive resource generation, substituting resource generations to be collected in a form of enemy bounties to progress through the game.

As a result of the switch, playtesting had to become more important, since enemy units' death is now the players only source of income. From the initial playtesting, it was shown that EXP was generally less restrictive than gold. Large enemy waves could provide sufficient EXP for era progression, particularly when Meteor Strike was used effectively. Gold, however, became the chokepoint for players progression, especially in the early game, where there was limited resource to purchasing units, turret holders and turrets. Moreover, since gold can only be obtained from enemy units, playing the wrong moves early game could be detrimental since there were cases where all the player units were destroyed, leaving them with nothing left to defend their base with.

To prevent such situations from happening,enemy gold rewards were therefore adjusted more heavily than EXP rewards. Likewise, adjustments for the starting gold amount was heavily tested to find out what is an optimal configuration such that it can still be difficult without making it impossible to win.

The images below are the Economy Values that we felt were good enough to make the game playable, without making it too easy or difficult. As mentioned in the previous chapters, there is an economy manager that will contain the economySystem script, which contains all the gold and exp tracking, and it does the transactions of gold and exp changes. On the other hand, the Player Spawner script will contain the era evolution cost as well as the special ability (Meteor Shower) cost for each era, allowing it be to scalable if needed to.



Another thing to note is as mentioned, the Passive Generation in this Milestone 3 is set to off, but if the developer wishes to allow passive generation in the future, instead of needing to rewrite the entire code again, he just need to allow passive to be on, making the transition entirely seamless and ensuring long term maintainability.

13.7.5. Player Unit Balancing

Diving into the details of the player units', here are the specifications we established for each one, ensuring they can effectively carry out their designated roles that we feel matches the unit. Note that the Movement Speed and Attack Cooldown values are distance per second(frame) and second(frame) respectively. The same is true for the training time, where all time related fields are using the same "clock" to ensure consistency throughout the game.

Player Unit	Health Points	Movement Speed	Damage	Attack Cooldown	Gold Cost	Training Time	Intended Role
Slime	110	1.45	12	1.1	50	1	Cheap cost unit used for early pressure and numbers.
Orc	150	1.25	18	1	90	2	Mid tier unit with greater durability than the Slime.
Skeleton	260	1	25	0.85	125	3	A hybrid tank unit that performs best with support. This is to ensure all era 1 units do not get eliminated too fast.
Swordman	200	1.15	26	1	120	2	Balanced melee unit with reliable damage and survivability.
Player Archer	150	1.1	32	0.9	160	3	Ranged unit (glass cannon) that is capable of attacking safely from behind allied units.
Magic Swordman	350	1.2	42	0.8	220	4	Stronger unit designed to challenge durable (high HP) enemies.

As mentioned in the intended role section, one recurring issue involved the Skeleton being eliminated too quickly by the Golem and ranged enemies. Its values, mainly Health Points (HP), therefore required adjustment so that it remained useful without becoming stronger than the Golem.

Ranged units also required careful tuning. Their ability to attack from behind allied frontline units meant that their effective damage could become disproportionately high when several were deployed together.

For this reason, while I initially considered only one to one unit interactions, I had to change up my approach where units were tested not only in one versus one encounters but also in mixed formations (To simulate a more realistic encounter).

13.7.6. Enemy Unit and Reward Balancing

In terms of the enemy statistics configurations, they were balanced along with their resource rewards, whereby stronger enemies needed to provide greater rewards so that difficult encounters remained worthwhile.

Enemy Unit	Health Points	Movement Speed	Damage	Attack Cooldown	Gold Reward	EXP Reward	Spawn At Start	Spawn Interval	Intended Role
Fire Mage	130	1.25	18	1.1	60	25	Yes	15	Enemy normal melee unit.
Enemy Archer	100	1.15	20	0.9	150	30	Yes	20	Enemy Glass Cannon, painful if left alone
Golem	400	0.6	24	1.1	250	80	No	40	The enemy tank unit, where has huge Health Point but slow speed.
Fire Giant (Boss)	950	0.65	45	1.2	1000	200	No	200	Boss Spawn Unit. Difficult to kill, but massive reward.

During playtesting, the Golem was initially too durable relative to the available player units, especially comparing it to era 1 (Undead) units. This created situations where the player had few effective responses other than Meteor Strike. Therefore, to ensure the difficulty is still there, without making the game impossible to clear, its health, damage and spawn timing were adjusted alongside the strength of the Skeleton and Magic Swordsman.

Likewise, we decided that the Boss spawn time should be set to 200 (frames / seconds) which should equate to 5 minutes, where by that stage of the game, if the player played his cards right, would have the resources needed to defeat the enemy boss to gain its massive rewards.

Overall, during this whole process, enemy statistics and spawn intervals were reviewed thoroughly so that stronger enemies would not appear before the player had a reasonable opportunity to prepare.

13.7.7. Turret and Special Ability Balancing

Last but not least, we also need to configure the turret (self defence) and special ability aspects of the game, whereby for the turrets, since it was hard to find other turret sprites that were free, we decided to configure the same crossbow turret to have different purposes for different eras.

Turret	Damage	Attack Cooldown	Attack Range	Gold Cost	Intended Role
Basic Crossbow	16	1.2	9	100	Affordable early defence support.
Rapid Crossbow	10	0.6	9	200	Slightly faster and still affordable early defence support.
Heavy Crossbow	35	1	11	500	Strong turret that does more damage after era progression.
Rapid Heavy Crossbow	25	0.3	13	1000	Most Expensive, late game defence option that is only available after era progression.

During the gameplay testing, turret construction costs were balanced together with turret holder upgrade costs. This prevented defensive structures from becoming the automatic best investment while still allowing them to protect the base during large enemy waves.

Moving on towards the special ability (Meteor Strike), the ability had to be configured as a powerful emergency ability rather than a normal attack.

Ability Variable	Final Value	Purpose
Experience (EXP) Cost	Era 1 (Undead): 250 Era 2 (Medieval): 500	Forces the player to choose between evolving to the next era or using the ability, making the gameplay more complex.
Number of Meteors	Era 1 (Undead): 60 Era 2 (Medieval): 125	Controls the size and reliability of the meteor shower hitting the target
Each Meteor Damage	50 (For both eras)	To ensure that a meteor strike if intended, be able to destroy enemy units in one hit.
Explosion Radius	2.5 (circular range)	Provides area damage against grouped enemies.

During early playtesting, repeated Meteor Strikes could clear the battlefield too easily and allow damage to the enemy base directly, making the game become too easy to clear. The ability was therefore restricted to enemy units, ensuring that normal unit deployment remained necessary to win the game.

13.7.8. Challenges faced during the building process of this feature

1. **<u>Balancing Gold and EXP Progression</u>**

Playtesting revealed that EXP was generally available in sufficient quantities, especially when Meteor Strike was used against large enemy waves. Gold, however, frequently prevented the player from producing units or constructing additional turrets.

Rather than increasing passive gold generation (since we decided to remove it), enemy gold rewards were adjusted. This allowed successful combat to fund future purchases while preventing passive waiting from becoming the safest strategy.

Overall, the main challenge faced was the extensive trial-and-error process required to fine-tune unit values. This task proved far more complex than initially anticipated. My original design assumed simple one versus one matchups. However, as the actual gameplay features a much higher level of systemic complexity, adapting to these multi-unit interactions revealed a slight gap in my initial foresight, which I had successfully navigated through countless cycles of intensive iterative testing.

13.7.9. Overall Outcome:

All in all, by testing and balancing the unit's interactions in game (unit strength, resource rewards, turret costs and progression requirements), we felt that this has helped improve the overall consistency of the game.

Each unit now has a clearer combat role, with stronger enemies providing more appropriate rewards, and the player must make meaningful decisions between unit production, turret investment, Meteor Strike usage and era progression.

While further adjustments could still be made through a larger external playtesting group, the current values provide a functional and strategically varied gameplay experience within the intended scope of the project.

13.8. Reflection and Potential Extensions

13.8.1. Milestone 3 Reflection

For this milestone, we felt that our primary focus was transitioning from implementing isolated gameplay mechanics to integrating them into a more cohesive, maintainable game system.

While earlier milestones focused on validating fundamental mechanics, such as unit spawning, combat, economy management, and turret attacks, Milestone 3 shifted more toward refining system design. We introduced queue-based production, dynamic era data, a Meteor Strike ability, improved turret interactions, automated testing, and a redesigned gameplay interface.

One of the main lessons we learned during this milestone was that adding features without considering the underlying architecture can quickly increase code complexity.

For example, PlayerSpawner originally handled unit spawning, economy checks, cooldowns, era progression and several user interface updates within a single script. As the project expanded, responsibilities had to be separated into dedicated systems and helper classes.

Ultimately, adopting structured data classes, centralised refresh methods, and explicit state management significantly enhanced the project's overall organisation and long-term maintainability.

13.8.2. Key Technical Lessons Takeaways

Separation of Responsibilities

Throughout this project, we recognised the importance of using dedicated manager states to control distinct game subsystems rather than centralising all logic into monolithic scripts. Furthermore, we realised that single scripts don't need to manage every data value directly. Instead, we could introduce helper classes that allow us to separate concerns and improve overall readability.

For instance, delegating financial logic to an EconomySystem script, queue operations to a UnitProductionQueue class, and targeting behavior to a TurretManager object significantly reduced clutter. As a result, each component became far easier to understand, modify, and test.

Data Driven Design

The introduction of PlayerEraData and TurretEraData classes demonstrated the clear benefits of separating configurable gameplay parameters from execution logic. Rather than embedding conditional checks for every distinct era, gameplay systems dynamically query the active data object. This structure enables the same core code to drive varying unit stats, turret interactions, costs, and visual assets across different eras.

Additionally, the introduction of the UnitProductionQueue class provided a tailored First-In-First-Out (FIFO) queue structure, allowing direct access to the contents for UI previewing and order management (like automatically start training immediately if no unit is currently being trained).

Therefore, we felt that by adopting a data-driven approach during early feature ideation, we are able to significantly improve scalability and allow our codebase to be easily understood for future developers to navigate and extend.

State Based Interaction Design

The redesigned turret system demonstrated the importance of representing interaction states explicitly. How we managed to do so was by using Normal, Build and Sell modes that prevented conflicting inputs and gave the interface a clear method of determining which actions and overlays should currently be displayed. As a result, this not only makes the User Interface cleaner for the players, but also ensures that there is no change for logical issues where multiple “modes” were allowed to be overlay onto one another.

User Interface Feedback

The changes in turret overlays, and the introduction of queue counters, progress bars and floating refund text demonstrated that a gameplay action should not only work correctly but should also communicate its result clearly to the player. This is because the improvement of visual feedback makes the system easier to understand without relying on extensive instructions.

Testing and Defensive Programming

Implementing automated testing allowed us to validate individual components in isolation without needing to load full scene data or run the entire game loop, saving significant time and resources during development.

To complement this, as mentioned previously, we introduced validation checks, read-only properties, and testing helper methods to ensure that data objects contain valid values before execution. These defensive programming practices make system behavior far more predictably, preventing silent errors and ensuring that developer oversight is caught early in the pipeline.

13.8.3. Scope and Current Constraints

Although the project has achieved the majority of the objectives outlined in our original proposal, several planned features were intentionally deferred. One such feature was a more advanced enemy AI capable of dynamically adapting its behaviour based on the player’s current strategy, resource management, and battlefield state.

However, as development progressed, we found that implementing comprehensive automated testing and balancing the overall gameplay required significantly more effort than initially anticipated. Developing reliable unit and integration tests, restructuring gameplay systems to improve maintainability, and repeatedly refining unit statistics through extensive playtesting, ultimately became larger engineering tasks than expected.

Therefore, after deliberation, rather than continuing to increase the project's feature count, we consciously prioritised improving the quality of the existing systems. We believed that delivering a stable, maintainable, and thoroughly tested codebase was more valuable than introducing additional mechanics that had not been sufficiently validated. Moreover, we felt that this decision also better reflected the software engineering practices emphasised throughout Orbital, where writing reliable, scalable, and maintainable software is just as important as implementing new functionality.

13.8.4. Potential Extensions

With the underlying architecture now significantly more modular through the introduction of dedicated manager classes, data driven era systems, and improved separation of responsibilities, we feel that the project is well positioned for future expansion without requiring major architectural changes.

One natural potential extension for future implementations would be the introduction of a more advanced enemy AI capable of analysing the player's current army composition, turret placement, available resources, and era progression before deciding whether to attack aggressively, defend, or evolve its own technology. This was originally considered during the planning stage but was ultimately postponed in favour of improving software quality and gameplay stability.

Beyond artificial intelligence, the existing architecture also allows additional eras, playable units, defensive structures, special abilities, enemy types, and maps to be introduced with relatively little modification to the existing gameplay systems. Moreover, we feel that stacking on these features would substantially boost replayability and player engagement.

Lastly, we should introduce larger scale community playtesting to provide valuable feedback for further gameplay balancing, difficulty adjustments, and quality of life improvements, since the current testing was mainly done by one person.

13.8.5. Overall Conclusion

Overall, we truly believe that the project successfully achieved its primary objective of developing a fully playable Age of War inspired strategy game while applying sound software engineering principles throughout the development process.

Across the three milestones, the project evolved from a basic combat prototype into a complete game featuring resource management, era progression, queue based unit production, dynamic user interface updates, turret construction, special abilities, gameplay balancing, and automated testing. More importantly, we developed a much stronger appreciation for designing systems that are modular, maintainable, and testable rather than simply continuing to add features.

Although several planned extensions, such as advanced enemy AI, remain as future work, we believe the decision to prioritise software quality, maintainability, automated testing, and gameplay refinement ultimately produced a stronger final product. The project not only fulfilled our original learning objectives but also provided valuable experience in applying object-oriented design, iterative development, testing methodologies, and software engineering practices to a real world application.

Last but not least, this project gave us a much greater appreciation for the game development industry. At the beginning of Orbital, we viewed game development primarily as implementing gameplay features. However, throughout the project, we realised that building an enjoyable game involves far more than simply writing code. Designing intuitive user interfaces, balancing gameplay mechanics, maintaining a clean software architecture, writing automated tests, debugging unexpected interactions between systems, and continuously refining the overall player experience all required significantly more time and effort than we had originally anticipated.

Therefore, while our project represents only a small scale prototype compared to commercial titles, the experience gave us a deeper respect for professional game developers and the complexity involved in creating polished games.

14. References

Age of War, a free flash action game! | *Max Games*. (n.d.). https://www.maxgames.com/game/age-of-war.html?

Alex-Morgan. (2026, June 4). *Fantasy Adventure Quest* | *Royalty-free music*. Pixabay. https://pixabay.com/music/adventure-fantasy-adventure-quest-537478/

ChatGPT. (n.d.). ChatGPT (Used for Art Generation Only). https://chatgpt.com/

Gatsby. (2023, June 22). *Simple Health bar Unity tutorial* [Video]. YouTube. https://www.youtube.com/watch?v=lYZayXViTN8

How to Make a Main Menu in Unity | *Beginner Tutorial*. (n.d.). YouTube. https://www.youtube.com/playlist?list=PLIvwrsXuTVRCMOtbhN-oRf8U8wba-Y0t7

Igara Studio S.A. (n.d.). *Aseprite*. Animated Sprite Editor & Pixel Art Tool. https://www.aseprite.org/

Ksjsbwuil. (2026, April 9). *Digital Click - 3* | *Royalty-free Music*. Pixabay. https://pixabay.com/sound-effects/technology-digital-click-3-513897/

PaulYudin. (2023, July 17). *War Game* | *Royalty-free music*. Pixabay. https://pixabay.com/music/main-title-war-game-158615/

Play Age of War 2, and more Action Games! | *Max Games*. (n.d.). https://www.maxgames.com/play/age-of-war-2.html

Technologies, U. (n.d.). *Unity - Manual: Unity 6.5 User Manual*. https://docs.unity3d.com/Manual/index.html

UI Sounds pack 4 (2) [For Hover button audio] | *Royalty-free Music*. (2025, June 15). Pixabay.

https://pixabay.com/sound-effects/film-special-effects-ui-sounds-pack-4-2-359741/

Unity Asset Store. (n.d.). https://assetstore.unity.com/